

Agent 评估实战课

Agent评估全景图——用一场考试看懂整个体系

学习目标：用一场驾考比喻，把 Agent 评估的 8 个核心概念映射到你熟悉的东西上，建立整门课的全景认知。

引言

Agent 开发已从“代码逻辑”变为“概率行为”——你改的不是代码，你改的是概率分布。

如果你只给 Agent 跑通了一次测试就敢上线——你是在赌博。

这不是比喻。大多数 Agent 开发者评估 Agent 的方式是：本地跑了几次，感觉还不错，上线。然后等用户投诉。

这门课教你用证据替代运气。

💡 为什么 Agent 评估需要一门专门的课？

因为 Agent 不是一个函数。

传统软件的测试逻辑很简单：固定输入 → 预期输出。写一次测试，跑一次，通过，提交。

Agent 打破了这个前提。**同样的输入，每次跑出来的结果可能完全不同。**你的 Agent 今天回答“订单已发货”，明天同样的问题可能回答“联系快递员”，后天可能直接退款。

这不是 Bug，这是 Agent 的本质特征。

而正因为这个特征，你需要一套全新的评估方法论。

驾考比喻：先认识 3 个最核心的概念

我发现学评估最难的不是技术，是语言。但我们不着急着背 8 个——先记住 3 个：

评估概念	驾考对应	一句话解释
Task	侧方停车（考试题目）	你让 Agent 完成的具体任务
Trial	你考了第三次（每次独立尝试）	同一个 Task 的单次执行
Grader	考官（评分逻辑）	判断 Agent 表现好坏的规则

完整的 8 个概念（Trace、Outcome、Harness、Suite、Ground Truth）会在 Ch03 详细展开——那时候你已经亲手跑过盲测，这些概念不再是术语，而是对你实践动作的命名。

课程六段路

这门课带你走六段路，每段解决一个核心问题：

第一篇·认知篇（Ch1–5）：别学概念，先动手。你先跑一次盲测，跑出差异了，再用专业语言描述你看到的東西。

第二篇·方法篇（Ch6–11）：从手工标注到代码规则到 LLM 裁判，搭三层自动化评分流水线。

第三篇·策略篇（Ch12–17）：搞清楚你评的是哪类 Agent，然后选策略、叠防御、定门禁。

第四篇·工程篇（Ch18–22）：从评测集、用例、基线到 Trace，把评估系统真正搭起来。

第五篇·进阶篇（Ch23–26）：应对长上下文、非确定性和 Bad Case 系统化反推。

第六篇·案例篇（Ch27–30）：看阿里云、腾讯 TPerf、AIGC 配图三个真实团队怎么踩坑、怎么填坑。

本节核心观点

一句话定调：Agent 开发已从“代码逻辑”变为“概率行为”——你改的不是代码，你改的是概率分布。

评估不是负担。评估是你唯一能信任的镜子。

实操

现在做两件事：

1. 写下你手头那个 Agent 的名字，和它要解决的核心任务（一句话）。
2. 回忆最近一次你感觉”Agent 好像不太对，但说不上来哪里不对”的输出——把那个输出写下来。

第一件事是贯穿这门课的”你自己的案例”。第二件事是贯穿这门课的”你自己的问题”——学到后面你会发现，那个”说不上来”的地方，正是评估体系要捕捉的东西。

下节预告

下一章，我们直接跳过理论——你去打开你的 Agent，同一个任务跑 3 次，看看发生了什么。

盲测你的Agent——同一个任务跑3次

学习目标：通过亲手盲测，直觉感受 Agent 的不确定性，建立“评估从实践出发”的认知基础。

引言

别急着学理论。先打开你的 Agent，同一个问题问 3 遍。

看看 3 次的答案——一样吗？

如果不一样，你之前是怎么判断它“好不好”的？

盲测三步

不需要任何工具，不需要任何框架。就是这三步：

第一步：选一个高频任务

选你的 Agent 最常处理的那类请求。客服 Agent 选一个典型咨询，代码 Agent 选一个常见的代码问题，研究 Agent 选一个常见的查询。

第二步：用完全相同的输入跑 3 次

复制粘贴。不要改一个字。在完全重置的对话窗口里跑。

“完全重置”在不同架构下操作不同：API 型 Agent → 新建 API Session，不携带任何历史上下文；Chat 界面型 → 点“新对话”，不要在已有对话里追加；有状态工作流型 → 重置所有中间状态（数据库、临时文件、会话缓存）。关键是：**每次 Trial 必须像 Agent 第一次见到这个任务一样。**

第三步：记录每次的结果差异

不只是记最终答案，记整个过程：

- 答案是否相同？
- 调用了哪些工具？顺序一样吗？
- 推理路径（如果能看到）是否一样？
- 有没有一次成功、一次失败？

真实案例：客服 Agent 的三次作答

一个客服 Agent，用户问”我的订单怎么还没到？“。完全相同的输入，跑 3 次：

第一次：查物流 API → 返回”已发货，在途中” → 回复”您的快递预计明天到达”

第二次：查物流 API 超时 → Agent 跳过了重试逻辑，直接进了退款分支 → 回复”已为您申请退款”

为什么超时会触发退款？ 物流 API 超时触发了 Agent 内部的一个异常分支——当 API 在 3 秒内无响应时，默认走了”订单异常处理”流程，而这个流程的第一步就是退款。跟物流本身没关系，是异常处理逻辑的问题。

第三次：查物流 API → 返回成功 → 回复正确 → 顺便推荐了几款新品

同一个问题，三个完全不同的答案。哪一个是对的？

这就是 Agent 的真实面目。

关键认知

如果你只跑了一次，你不会知道你的 Agent 还有另外两种行为。

更可怕的是：你上线之前只跑了一次，结果通过了——那说明你上线的是三种行为里最好的那次，还是随机的一次？你不知道。

这就是为什么”跑通一次”不等于”Agent 好用”。

本节核心观点

盲测不是正式的评估方法，但它能做到一件事：**让你用自己的眼睛看到 Agent 的不确定性。**你可以在不学任何理论的情况下，在 10 分钟内完成第一次 Agent 评估。你刚做的就是评估——只是还没给它起名字。

实操

现在就做：

1. 选你 Agent 的一个高频任务
2. 固定输入，跑 3 次
3. 记录：3 次结果是否一致？中间步骤是否一致？有哪次是你没想到的行为？

把结果保存下来。Ch02/03/04/05 认知篇的四章都会围绕这 3 次结果展开分析。

如果 3 次结果完全一样怎么办？ 这说明你的 temperature 可能设为了 0，或者任务过于简单。建议换一个边界输入试试——模糊的表达（“帮我看一下那个事”）、缺失的信息（不提供订单号）、带情绪的语气（“你们到底行不行啊”）——这些更容易暴露 Agent 的行为差异。

下节预告

下一章，我们分析你刚跑出来的差异——为什么 3 次结果不一样？这些差异背后是 3 个 Agent 评估的底层挑战。

为什么3次结果不一样——Agent评估的三大独特挑战

学习目标：理解 Agent 非确定性的底层原因，把 Ch1 的盲测差异映射到三个专业挑战上。

引言

传统软件测试有个美丽的前提：同样的输入，永远得到同样的输出。

你刚跑完的盲测已经证明——Agent 的世界里没有这个前提。

这不是 Bug。这是 Agent 的本质特征。

三大独特挑战

你在 Ch1 看到的三次不同结果，不是随机偶然。它们来自三个系统性原因。

挑战 1：非确定性

LLM 本身就是个概率系统。

自回归生成的链条是这样的：每次生成下一个 token 时，模型先算出所有可能 token 的得分（logits），然后做温度缩放和概率转换，最后从概率分布里随机采样。

就像每次掷一枚偏重的骰子——你改的是骰子的重量分布，不是某次的结果。

一次 Agent 任务可能涉及几十次这样的采样——每次的微小差异通过链式法则指数级放大。

所以“跑通一次”不代表“稳定能跑”。你没见过的那两种行为，一直都在。

挑战 2：黑盒化

你看到的是输入和输出。中间发生了什么，你不知道。

更可怕的是：模型升级、Prompt 微调、工具链变化——任何一个都可能导致行为漂移，而且肉眼难以察觉。

举一个真实的场景：你把 system prompt 里的”请礼貌回复”改成”请简洁回复”——你觉得只是改了一个措辞。结果某类用户的投诉率上升了 20%，因为”简洁”在投诉场景下被理解成了”冷漠”。但你没有打开过任何日志，你甚至不知道这件事发生了。

这就是黑盒化的危险：你改了，你以为你改了，但你不知道它变了。

挑战 3：错误级联放大

一次任务涉及的工具调用可能多达几十步。

第一步的小偏差，传递到第二步，放大。传递到第三步，继续放大。等到最终结果出来，你已经不知道问题出在哪了。

客服 Agent 那次”直接退款”就是这样发生的——不是因为退款逻辑出了问题，是因为物流 API 超时触发了一个异常分支，而那个分支没有重试机制。

🤖 对比：传统软件 vs Agent

维度	传统软件	Agent
输入→输出	固定输入 → 固定输出	相同输入 → 不同输出
失败可见性	报错明确，栈追踪清晰	行为偏差，无报错
测试策略	写一次，跑通，提交	多次运行，统计证据
优化方式	修代码	改概率分布

这不是”Agent 有缺陷”。这是 Agent 的本质——你改的不是代码，你改的是概率分布。**这意味着你不能再靠”看代码逻辑”来判断 Agent 好不好——你只能靠统计数据。**

关键认知

如果你只跑了一次，你看到的是概率分布的一个样本，不是 Agent 的真实能力。

这就是为什么 Ch1 的盲测要跑 3 次——你需要的不是一个答案，你需要的是一个分布的样子。

非确定性不是需要消灭的敌人。它是需要管理的现实。

本节核心观点

三个独特挑战： – **非确定性**：链式随机放大，跑通一次 $\neq$ 稳定能跑 – **黑盒化**：行为漂移肉眼不可见 – **错误级联**：前序偏差沿链路放大，溯源困难

这三个挑战解释了你在 Ch1 看到的一切差异。

实操

回看 Ch1 的 3 次盲测结果，逐一标注：

- 哪次差异是**非确定性**造成的？（同一逻辑，随机结果不同）
- 哪次是**黑盒行为**？（中间步骤发生了你没预期的事）
- 有没有**错误级联**？（前面一步出问题，后面跟着崩了）

示范标注：以 Ch1 客服 Agent 为例——Trial 2 中”物流 API 超时→直接退款”是错误级联，因为超时是触发条件，导致后续的异常分支被激活。Trial 3 中”物流正确但多推荐了新品”是黑盒行为——你不知道 Agent 为什么在物流查询场景里调用推荐 API。

把标注结果保存下来。后面方法篇的所有工具，都是为了处理这三个挑战。

下节预告

下一章，我们把 Ch1 做的事映射成专业语言——Task、Trial、Trace、Grader……你已经把它们全部实践了一遍，只是还没给它们起名字。

评估核心概念——把你刚做的事映射成专业语言

学习目标：把 Ch1 盲测的动作映射到 8 个评估专业概念，建立评估语言体系。

引言

你去看 Anthropic 的评估文章，会被 8 个概念砸晕。

Task、Trial、Trace、Outcome、Grader、Harness、Suite、Ground Truth——读不到两分钟就头晕了，然后关掉，继续靠感觉开发。

但你刚在 Ch1 已经把它们全部实践了一遍。只是还没给它们起名字。

8 个概念的真面目

用你 Ch1 的客服 Agent 盲测逐一对应：

Task（任务）：用户问“我的订单怎么还没到？”——这就是你让 Agent 完成的具体任务。一个 Task 定义了输入是什么、期望输出是什么。

Trial（单次执行）：你跑了 3 次，每次叫一个 Trial。Trial 1 查到物流正常，Trial 2 直接退款，Trial 3 推荐新品。每次都是同一 Task 的独立实例。

Trace（执行记录）：考官手里的评分表——Agent 查了哪个 API、参数是什么、返回了什么、下一步做了什么。整个决策链条的完整记录。不是最终结果，是过程。

Trial 1 在 Langfuse 里记录的 Trace 长这样：

```
{
  "name": "客服Agent-物流查询",
  "input": "我的订单怎么还没到?",
  "output": "您的快递预计明天到达, 请耐心等待",
  "latency_ms": 1948,
  "spans": [
    {
      "name": "llm: 意图识别",
      "output": { "intent": "logistics_query" },
      "tokens": 188,
      "latency_ms": 891
    },
    {
      "name": "tool: logistics_api",
      "input": { "order_id": "12345" },
      "output": { "status": "in_transit", "eta": "明天" },
      "status": "success",
      "latency_ms": 423
    },
    {
      "name": "llm: 生成回复",
      "output": "您的快递预计明天到达, 请耐心等待",
      "tokens": 117,
      "latency_ms": 634
    }
  ]
}
```

这就是 Trace 的样子。每个 `span` 是一个执行步骤。评估时会用到不同的层：

字段	评估用途
根级 <code>output</code>	Outcome——结果评分器读这里
<code>spans[].name</code> + <code>input/output</code>	工具调用路径——过程评分读这里
<code>spans[].status</code>	异常检测——有没有超时、报错
<code>spans[].latency_ms</code> / <code>tokens</code>	效率评估——绕没绕路、花了多少

只看 Outcome，是看冰山露出水面的那一角。Trace 是整座冰山。

Outcome（最终输出）：Agent 最后给用户的那句话。“您的快递预计明天到达”就是 Trial 1 的 Outcome。

Grader（评分器）：你判断”这个回复对不对”的逻辑。Ch1 里你用的是人脑，后续我们会把它写成代码或 AI。

Agent Harness（评估框架）：你跑 Agent 的那个环境——是本地命令行、还是 API 调用、还是专门的测试平台。它负责启动 Agent、注入输入、捕获输出。**你在命令行里跑的那个脚本就是最简单的 Harness。**

Suite（评估套件）：一组 Task 的集合。你有 50 条客服问题的 test.csv——这就是一个 Suite。比如”客服 Agent 的 50 种常见询问”就是一个 Suite。

Ground Truth（标准答案）：你手工标注的”正确答案”——用来校准 Grader 的基准。

一个公式串起来

这 8 个概念的关系，一个公式就够了：

```
Eval = 输入 Task
      → Agent Harness 执行
      → 产出 Trial (= Trace + Outcome)
      → Grader 判断
      → 可对比的分数
```

多个 Task 组成 Suite。Grader 拿 Outcome 和 Trace 对比 Ground Truth，输出分数。

这不是新知识。这是你 Ch1 动作的专业语言版本。

对照：Ch1 的你 vs 现在的你

Ch1 你的动作	现在的专业名称
选一个高频问题	定义一个 Task
跑 3 次	执行 3 次 Trial
记录每次过程	捕获 Trace
记录最终答案	记录 Outcome
脑子里判断对不对	运行一个 Grader
3 次合在一起分析	运行一个 Suite
你认为是“正确”长什么样	Ground Truth

你已经完成了一次完整的 Agent 评估。

本节核心观点

8 个概念不是凭空发明的——它们是评估动作的精确命名。

有了名字，你才能把“感觉跑了几次”升级成“系统化的评估流程”。

语言是工具。工具精确，思维才精确。

实操

把你 Ch1 的盲测映射到 8 个概念：

1. 你的 Task 是什么？（一句话描述）
2. 一次 Trial 结束时，Trace 里有哪些字段？（工具调用、参数、返回值……）
3. 你的 Grader 是什么？（目前大概率是你脑子——写下来，变成文字）
4. 你的 Ground Truth 是什么样的？（“正确的回复应该包含哪些要素？”）

把这四个答案写下来。后面的实操全部在这个基础上迭代。

下节预告

下一章，我们扩展 Grader 的视野——不只是评”最终答案对不对”。一个 Agent 可能答案对了、但过程差点删库，这算通过吗？

不只评对不对——五大评估维度

学习目标：理解 Agent 评估不只看”对不对”，掌握五大评估维度框架，学会从多角度审视 Agent 表现。

引言

评估和考试不一样。

考试只看最终答案——答对了，满分。Agent 评估还得看你是光明正大做出来的，还是差点把数据库删了但侥幸答对了。

你在 Ch1 判断 Agent 好不好时，大概率只看了一件事：最终答案对不对？

这是最大的评估盲区。

先理解优先级框架

在展开五个维度之前，先建立阅读框架——P0/P1/P2 决定了每个维度的权重：

优先级	维度	含义
P0	功能正确性、鲁棒性	不能妥协，任何 bad case 都是 blocker
P1	过程质量、效率	影响稳定性和成本，需要系统优化
P2	体验对齐	影响用户感受，可迭代改进

P0 只看一次就通过是盲目自信。P2 永远第一个被抱怨。

五大评估维度

用 Ch1 的客服 Agent，对同一次输出从五个维度重新打分：

维度 1：功能正确性 (P0)

Agent 做对了该做的事吗？

Trial 1: 查物流 API，返回”已发货”，回复”您的快递预计明天到达”——正确。

Trial 2: 物流 API 超时，直接退款，没有经过用户确认——**不正确**，触发了不该触发的行为。

P0 是最高优先级。功能不对，其他维度免谈。

维度 2：过程质量 (P1)

Agent 是怎么做到的？路径合理吗？

Trial 3: 物流查询返回正确，但 Agent 顺便推荐了几款新品。

最终结果对了。但这个”顺便推荐”是合理的行为吗？在用户焦虑地查快递的场景里——不合理。

过程质量暴露的问题，结果评估永远看不到。从 Trace 里就能发现：

```
{
  "spans": [
    { "name": "tool: logistics_api", "status": "success" },
    { "name": "tool: recommendation_api", "input": { "scene": "物流查询" }, "status": "success" },
    { "name": "llm: 生成回复", "output": "您的快递预计明天到达，顺便推荐您..." }
  ]
}
```

Outcome（最终回复）显示”查询成功”，结果评分器会给高分。但 `spans[1]` 里那个 `recommendation_api` 调用——多余的、不合时宜的——只在 Trace 里可见。

结果评估看到的是最后一个 span 的 output；Trace 评估看到的是中间所有不该有的步骤。

维度 3：鲁棒性 (P0)

遇到异常情况，Agent 还能正常工作吗？

Trial 2 的根本问题不是”直接退款”——是物流 API 超时后，Agent **没有重试机制**，直接走了一条危险的异常分支。

P0 级别的问题。注意：Trial 2 这一个事件同时击穿了功能正确性（未经确认就退款）和鲁棒性（超时后无容错）——这就是多维评估的价值，一个失败同时暴露两个维度的问题。

生产环境里，异常才是常态。

维度 4：效率（P1）

Agent 用了多少资源完成任务？

Trial 3 多调了一次推荐 API，增加了延迟，消耗了更多 Token。

任务完成了，但成本上升了。效率不是“能不能完成”，是“用多少代价完成”。

维度 5：体验对齐（P2）

最终输出的语气和风格是否符合场景期望？

Trial 1 回复自然友好——好：“您的快递预计明天到达，如有问题随时联系我们 😊”。Trial 2 退款后的回复“已为您申请退款”语气生硬，没有道歉，没有解释——差。

好的体验对比：同一场景下，“已为您申请退款” vs “抱歉给您带来不便，已帮您提交了退款申请，预计 1-3 个工作日到账。在此期间如需帮助，随时找我”——后者让用户感受到被重视，而不只是被处理。

P2 优先级最低，但经常是用户投诉的直接来源。

关键认知

只看最终答案的评估，会漏掉两类问题：

1. **过程中的危险行为：**答案侥幸对了，但路径走过危险区域
2. **系统性脆弱点：**正常情况表现良好，异常情况直接崩溃

这两类问题的共同特征：**在你的测试集里看不到，在生产环境里最先爆发。**

五个维度，各自对应 Trace 里的不同字段：

评估维度	在 Trace 里看哪里	评分器类型
功能正确性	根级 <code>output</code> （最终回复）	代码 / LLM 裁判
过程质量	每个 <code>span</code> 的调用序列 + 参数	LLM 裁判 / 人工
鲁棒性	<code>status: error</code> 后的下一个 <code>span</code>	代码规则
效率	各 <code>span</code> 的 <code>latency_ms</code> + <code>tokens</code>	代码
体验对齐	最后一个 <code>llm</code> <code>span</code> 的 <code>output</code>	LLM 裁判 / 人工

后续方法篇会分别教你怎么写这几类评分器。先记住这张映射表——它是五维度和 Trace 之间的桥。

本节核心观点

五个维度，两套互补视角：

- 腾讯 TEG 偏结果导向：功能正确性 / 过程质量 / 效率 / 鲁棒性 / 体验
- 通用 Agent 偏过程导向：任务完成度 / 工具使用质量 / 规划推理 / 安全效率

实际建议：**两套对照使用，看结果用腾讯框架，看过程用通用框架。**

评估维度不是越多越好。P0 没过，P1/P2 的分数没有意义。

实操

回看 Ch1 的 3 次盲测结果，从五个维度各打一个主观分（0-5 分）：

	Trial 1	Trial 2	Trial 3
功能正确性			
过程质量			
鲁棒性			
效率			
体验对齐			

找出你最薄弱的维度。下一章我们把这个主观分变成可自动计算的指标。

下节预告

下一章，我们解决一个实际问题：主观分没法放进 CI/CD。怎么把”功能正确性感觉是 7 分”变成可自动运行的代码？

怎么算好——指标设计三原则

学习目标：掌握指标设计三原则（可量化→可自动评测→可归因），把主观判断转化为可进入 CI/CD 的客观指标。

引言

数字不等于质量。但没有数字，你什么都改不了。

有一个 Agent 团队定义“回复长度 > 50 字”为高质量标准——结果 Agent 学会把每句话加上无意义的客套和重复，把 30 字就能说清楚的事硬拉到 80 字。分数满分，用户投诉翻倍。

你量什么，Agent 就优化什么。所以要小心你量的是什么。

⚠ 主观分的死穴

你在 Ch4 给五个维度打了主观分——“功能正确性感觉是 7 分”。

这个分数的问题：

- 明天换个人打，得 5 分
- 放进代码里跑不了
- 分数低了，不知道该改哪里
- 分数高了，不知道是真的好还是侥幸好

主观判断是评估的起点，不是终点。

🔧 指标设计三原则

以客服 Agent 的“功能正确性”为例，一步步走：

原则 1：可量化

把抽象概念转为可计算的值。

“功能正确性感觉不错” → “物流单号是否与订单匹配？（是/否）”

不是形容词，是可以计算的判断条件。可量化的指标，不同的人/系统运行，得到同样的结论。

原则 2：可自动评测

支持自动化，才能融入 CI/CD。

“物流单号是否与订单匹配？” → `assert trace.tracking_id == order.tracking_id`

写成一段脚本，每次 Agent 跑完自动执行。不需要人盯着，不需要人工判断。

可自动评测 = 每次提交代码就能自动验证 Agent 没有退化。

原则 3：可归因

指标下降时，能定位到哪个模块出了问题。

物流单号匹配失败 → 排查顺序：接口层（物流 API 返回格式是否变了？） → 工具层（Agent 有没有正确传参？） → 模型层（LLM 有没有正确提取单号？） → Prompt 层（提取逻辑是否清晰？）。

可归因的指标，让优化有方向。不可归因的指标，只能让你焦虑。

三原则的递进关系

可量化 → 可自动评测 → 可归因

三个不是并列的，是递进的：

- 不可量化的东西，写不成代码
- 不能自动跑的代码，进不了 CI
- 没有归因能力的指标，降了也不知道改哪里

设计一个指标，按这三个问题逐一检查：这个值能算出来吗？能自动算吗？降了知道查哪吗？

客观指标 vs 主观指标

类型	举例	适用场景
客观	单号匹配准确率、工具调用次数、Token 消耗	P0 功能验证、效率监控
主观	回复是否自然、推荐是否合适	体验对齐、过程质量

两种都需要，最佳实践是组合。

客观指标把 P0 问题自动拦住，主观指标目前先用人工处理，方法篇 Ch8 会教你用 LLM-as-Judge 来自动化主观判断。

用客观指标管确定性问题，用主观指标管模糊问题——分工明确，互不替代。

本节核心观点

指标设计三原则递进：

- **可量化**：抽象概念转为可计算的值
- **可自动评测**：写成代码，进 CI/CD
- **可归因**：指标下降，能定位到哪个模块

一个通不过三原则的指标，要么没用，要么会让你越看越迷糊。

从你最薄弱的那个维度开始，按三原则设计第一个真实的评估指标。

实操

从 Ch4 五个维度中选你最薄弱的那个，按三原则设计一个指标：

1. **可量化**：这个维度的”好”如何表达成可计算的条件？
2. **可自动评测**：这个条件怎么写成 Python/Bash 代码？（伪代码也行）
3. **可归因**：如果这个指标降了，你会首先去查哪里？

写完之后检查：这个指标能进你的 CI/CD 吗？如果不能，卡在哪一步？

下节预告

认知篇结束。下一篇方法篇，从手工评估开始——你先建立 Ground Truth，再用代码规则替代人工，再用 LLM 当裁判，最后串成三层流水线。第一步：先让自己的判断变得精确。

先手工评——建立你的Ground Truth

学习目标：理解人工评估的三种用途，建立你的第一版 Ground Truth，为后续自动化评分奠定基准。

引言

人工评估是黄金标准。但黄金很贵——你的评估预算里，只有 5–10% 的样本配得上它。剩下的 90%，我们交给规则和 AI。

问题来了：规则和 AI 的对错，谁来判定？

还是得靠人。人工评估不是可以跳过的环节，而是整个体系的校准基准。你先建好地基，上面的楼才站得住。

💡 人工评估的三种用途

不是所有人工评分都是同一件事。三种用途，完全不同的目的：

用途 1：建立 Ground Truth

告诉所有后续评分器“正确答案长什么样”。

没有 Ground Truth，你的代码规则在检查什么？你的 LLM 裁判在向什么对齐？都是无根之木。

Ground Truth 是整个评估体系的起点，不是可选项。

用途 2：仲裁分歧

代码规则说“通过”，LLM 裁判说“不通过”——谁对？

这时候需要人来仲裁。不是为了裁判哪一方，是为了发现评估体系本身的问题。代码规则可能太严，LLM 裁判可能理解偏了，也可能这条样本本来就是边界案例。

仲裁的结论，应该作为新的 Ground Truth 样本写进去。

用途 3：发现盲点

只有人工标注时，你会问自己一个问题：“这个回复有问题，但我说不出在哪里。”

这种说不出的直觉，往往是评估体系还没有覆盖的维度。

小实操：找一条你打分时停顿超过 3 秒的输出，写下“我觉得有问题，因为___”。这个空格，就是你评估体系的下一个扩展方向——把它记下来，之后扩展成新指标。

客服 Agent 的第一版 Ground Truth

用 Ch1 的三条输出，手工标注：

Trial 1：物流查询正确，回复“您的快递预计明天到达”。

→ **通过**。理由：识别了用户意图，调用了正确工具，返回信息准确，语气自然。

Trial 2：物流 API 超时，直接退款，回复“已为您申请退款”。

→ **不通过**。理由：物流超时不应直接触发退款，应先重试或告知用户。退款是不可逆操作，必须经用户确认。

Trial 3：物流查询正确，回复准确，但额外推荐了新品。

→ **部分通过**。理由：功能正确，但在物流查询场景中主动推销不合适。判断“不通过”过重（功能没坏），判断“通过”不准（行为有问题）——记为边界案例。

这三条，就是你的第一版 Ground Truth。

标注的两种组织方式

专家评测：你自己或领域专家来标注。

优点：质量高，判断准确，一致性好。缺点：贵，规模上不去。适合建立初始 Ground Truth、处理高歧义样本。

众包评测：分发给多人标注，取多数意见或均值。

优点：规模可扩展。缺点：个体差异大，需要明确的标注规范。

怎么选？ Ground Truth 建立阶段用专家评测（质量优先），稳定运行后用众包验证（规模优先），边界案例和校准始终用专家评测。

核心原则：人类只用于建立标准和处理模糊案例，不做常规重复劳动。

一旦 Ground Truth 建立好，常规样本的评估交给代码和 AI。

Ground Truth 最简单的存储格式（CSV，每行一条）：

```
input,expected_behavior,label
"查订单12345的物流","返回物流状态+预计到达时间,不触发退款和推荐",通过
"查订单abcde的物流","订单号不存在,告知用户未找到,不崩溃",通过
```

不需要数据库，不需要框架。一个 CSV 文件 + 版本控制 = 你的第一版 Ground Truth。

本节核心观点

人工评估不是“你最终肯定要做的事”，是“你现在必须先做的事”：

- **建立 Ground Truth：**后续所有自动化评分的校准基准
- **仲裁分歧：**发现评估体系的问题，不是补丁
- **发现盲点：**把说不出的直觉变成新指标

没有 Ground Truth 的自动化评估，等于用一把没有刻度的尺量东西。

实操

对 Ch1 的 3 条 Agent 输出做人工标注：

1. 每条写“通过/不通过/部分通过”
2. 写下判断理由（1-2 句话，具体说明为什么）
3. 如果你犹豫了——记下来。那个犹豫本身就是信息：这条是边界案例，还是你的评估标准还没有清晰到能判断？

把这三条标注保存下来。后面章节的所有自动化评分，都要对齐这份 Ground Truth。

下节预告

你手工标了 3 条。现在有 300 条——你能把”手工判断的逻辑”写成代码，让它自动跑吗？下一章，代码评分器。

让规则替你打分——代码评分器

学习目标：掌握代码评分器的三种形式，把手工标注逻辑转为可自动运行的规则代码，并理解代码评分器的边界。

引言

代码评分器就像一个严格的会计——你多写了一个空格它都觉得你错了。但有时候，这种严格正是你需要的。

问题不在于它严不严，而在于：你知道在哪种判断上用它，在哪种判断上别用它吗？

💡 三种确定性方法

代码评分器的核心逻辑：把判断转为可执行的代码，给出 True/False 或数值。

方法 1：精确匹配与规则检查

最简单，最可靠。判断某个字段存不存在、值是不是对的、顺序对不对。

```
# 物流单号是否与订单匹配
assert trace.tracking_id == order.tracking_id

# 是否有未经确认的退款
assert not (has_refund_action and not user_confirmed)

# 工具调用次数是否合理
assert tool_call_count <= baseline_count * 1.5
```

这三条，直接把 Ch6 的人工标注逻辑翻译成了代码。

方法 2：语义相似度

不要求精确匹配，而是检查含义相近程度。BERTScore、Cosine 相似度。

适合检查”Agent 的回复是否包含物流信息”——用户说”顺丰”，你的 Ground Truth 里写的是”SF Express”，精确匹配会误判，语义相似度不会。

方法 3：二进制测试

输出一个二值结果：通过/不通过。

```
# 代码能编译吗
result = subprocess.run(['python', '-c', code], capture_output=True)
assert result.returncode == 0

# 文件是否生成
assert os.path.exists(output_file)

# Token 是否超限
assert token_count <= 4096
```

这类用于检查客观约束：有没有超时、有没有崩溃、输出文件存不存在。

把 Ch6 标注转为代码规则

Trial 1 的通过理由：“识别了用户意图，调用了正确工具，返回信息准确”。

转为代码：

```
def check_trial_1(trace, outcome):
    # 识别意图
    assert trace.intent == "logistics_query"
    # 调用了物流工具
    assert "logistics_api" in [step.tool for step in trace.steps]
    # 返回包含快递状态
    assert "快递" in outcome.text or "物流" in outcome.text
    return True
```

Trial 2 的不通过理由：“退款未经用户确认，且没有重试物流查询”。

```
def check_no_unauthorized_refund(trace):
    for step in trace.steps:
        if step.tool == "refund_api":
            # 退款前必须有用户确认
            assert step.user_confirmed == True
    # 如果物流超时，必须有重试
    if trace.logistics_timeout:
        retry_steps = [s for s in trace.steps if s.action == "retry_logistics"]
        assert len(retry_steps) > 0
    return True
```

Trial 3 的部分通过（边界案例）——代码规则在这里遇到了边界：

```
def check_no_unprompted_recommendation(trace, outcome):
    # 用户没有要求推荐
    if "推荐" not in trace.user_input:
        # Agent 不应该主动推荐
        assert "推荐" not in outcome.text # ← 这条规则太简单，会误判
```

这条规则写不好——“推荐”这个词在不同语境下含义不同。这就是代码评分器的边界。

⚠ 代码评分器的优势与边界

优势： – 毫秒级运行，零人力 – 100% 可复现：同样的行为，每次判断相同 – 可调试：看得从哪一步开始偏

边界：

对“合法但形式不同”的变体脆弱。Agent 说“纽约”，Ground Truth 里写的是“New York”——精确匹配判错，但实际是对的。

对语义模糊的判断无能为力。“这个推荐时机合不合适”——代码不知道。

识别代码评分器的适用范围：判断“有没有”，不判断“好不好”。

本节核心观点

代码评分器是三层评估流水线的第一层，处理所有“有明确对错”的判断：

- **精确匹配**：字段值、调用序列、参数准确性
- **语义相似度**：含义接近但形式不同
- **二进制测试**：客观约束（超时、崩溃、文件存在）

能写规则的不写 Prompt，能写代码的不请 LLM。这是评估效率的第一原则。

实操

把 Ch6 的 3 条标注，转成至少 2 条可运行的代码规则（Python 伪代码也行）：

1. 写出规则逻辑
 2. 对每条规则标注：它是“确定能判对的”还是“可能误判的”？
 3. 找出你的规则覆盖不了的那种判断——记下来，那是下一章 LLM 裁判要处理的部分
-

下节预告

代码规则处理不了“回复语气是否自然”、“推荐时机是否合适”这类判断。这时候你需要第二个裁判——用 LLM 来评 LLM。

让AI替你打分——LLM as Judge

学习目标：掌握 LLM-as-Judge 的核心设计，学会写出可靠的评估 Rubric，让 AI 自动处理代码规则判断不了的“软判断”。

引言

让 LLM 当裁判去评判另一个 LLM——这听起来像让一个学生批改另一个学生的试卷。能行吗？

能，但你不能让它“自由心证”。

你让 LLM 做任何事，都需要给它明确的指令。评估这件事更是如此——你不是要它发表感想，你是要它产出可以被机器处理的评估数据。

💡 代码规则覆盖不了什么

Ch7 的代码规则可以判断：物流单号是否匹配，退款是否经过确认，工具调用次数是否合理。

但有些判断，代码写不了：

- “这个推荐时机合不合适？”
- “Agent 在用户表达不满时，有没有表现出共情？”
- “回复是否自然友好，没有机器腔？”
- “解释是否清晰，用户能不能理解？”

这类判断需要语义理解和语境感知——这正是 LLM 擅长的。

设计一个有效的 Rubric

Rubric 是你给 LLM 裁判的评分标准。设计得好，评分稳定可信。设计得差，等于白跑。

用 Trial 3 举例（物流查询正确，但多余推荐了新品）：

Rubric 需要覆盖至少三个维度：

维度 1：功能正确性 – 5 分：物流信息完整且准确（包含当前状态 + 预计到达时间） – 3 分：信息部分正确（有状态但无时间预估） – 1 分：物流信息错误或缺失

为什么只有 5/3/1 三个档位？ 三档是为了逼迫评分者做明确判断——“差不多给 3”的模糊中间态能让评分器变得毫无区分度。如果真有 4 分的场景，它要么是 5（接近完美），要么是 3（有明显不足）。

维度 2：过程质量 – 5 分：只调用了必要工具，没有多余步骤 – 3 分：调用了多余工具，但不影响结果 – 1 分：调用了不该调用的工具（如未经请求的推荐）

维度 3：体验对齐 – 5 分：语气自然友好，内容与场景匹配 – 3 分：语气可接受，但内容超出场景 – 1 分：语气生硬或内容与场景严重不符

输出格式必须被限死

Rubric 设计好了，还有一步关键：**告诉 LLM 怎么输出**。

你不是要它写一篇评语，你是要它输出**结构化数据**。

```
{
  "pass": false,
  "total_score": 11,
  "dimensions": {
    "functionality": {"score": 5, "reason": "物流状态和预计到达时间均准确"},
    "process_quality": {"score": 3, "reason": "推荐 API 调用不必要，但不影响主功能"},
    "experience_alignment": {"score": 3, "reason": "语气友好但主动推销与查询场景不符"}
  },
  "evidence": ["Agent 调用了推荐接口，用户未要求推荐"],
  "flags": ["unexpected_recommendation"]
}
```

三个关键要求： 1. 整体是否通过 (bool) 2. 每个维度的分和理由 (支撑后续归因) 3. 引用了哪些具体证据 (防止 LLM 凭感觉打分)

这是评估数据，不是审稿意见。评估数据会被系统持续积累、分析趋势、触发报警。审稿意见读完就扔了。

🤖 五种 LLM 评估方法

不是所有场景都用同一种方法：

方法	适用场景	推荐度
Rubric-based scoring	多维度打分	★★★★ 首选，适合 90% 日常评估
Pairwise comparison	只需比较两个版本谁更好	★★★ 选型决策时使用
Multi-judge consensus	高风险决策，需降低单一偏差	★★★ 高风险场景补充
Natural language assertions	布尔判断 (是/否)	★ 简单场景，可用代码规则替代
Reference-based evaluation	有标准答案对照	★ 适合有 Ground Truth 的场景

推荐策略：日常迭代用 Rubric-based scoring；只需选版本时用 Pairwise；高风险决策时叠加 Multi-judge consensus。Natural language assertions 和 Reference-based 作为补充工具。

🎯 本节核心观点

LLM-as-Judge 处理代码规则处理不了的”软判断”，核心是两件事：

- **Rubric 清晰**：每个维度的 5/3/1 分对应什么，写清楚
- **输出格式固定**：JSON 结构，包含 pass/score/reason/evidence

关键技巧：在 LLM 裁判的 Prompt 中，要求它**先写 100 字客观描述，再打逐维度分**——实验表明，先描述再打分比直接打分的准确率高出约 20%（经验值，基于多组人工标注对照）。

你收集的是评估数据，不是评语。评估数据要能被系统读，不只是被人读。

另外，`confidence`（置信度）字段在 Ch10 的流水线中会被用到——对你可以在 JSON 输出中加入：同一输入跑 3 次 LLM 裁判，若三次总分极差超过 2 分，标记为低置信度，升级人工仲裁。

实操

为你的 Agent 设计一个 Rubric（至少 3 个独立维度），用这个 Rubric 对 Ch6 的 3 条 Agent 输出打分：

1. 写下每个维度的评分标准（5/3/1 分）
 2. 设计 JSON 输出格式
 3. 让 LLM 对 3 条输出打分，记录结果
 4. 对比 LLM 打分和 Ch6 的人工标注——哪里有分差？
-

下节预告

你让 LLM 打出了分数。但这个分数该长什么样——一个数字？一个标签？还是一组数字？三种组织方式，对应三种完全不同的决策场景。

分数怎么组织——GSB MOS Likert

学习目标：掌握 GSB、MOS、Likert 三种分数组织方式，理解各自适用场景，学会根据决策需求选择正确的评分框架。

引言

你让 LLM 打分，它给你一个 4.2。

4.2 到底是什么意思——比上一版好？整体不错？还是功能 5 分、体验 3 分？三种意思，三种完全不同的后续行动。

数字是答案，但你先得知道你在问哪个问题。

💡 三种方式，三种问题

GSB (Good / Same / Bad)

你在问：这个版本比上一版更好、一样，还是更差？

对比两个版本的 Agent 输出，每个维度三选一：

维度	新版 vs 旧版
物流信息准确性	Same
回复语气自然度	Better
额外行为合理性	Worse

输出结论：整体 Same with concern（一项变差了）。

GSB 最快，最适合日常迭代。你改了一版 Prompt，不需要知道它绝对有多好，只需要知道它比上一版好不好。

GSB 聚合规则：多维度出现混合结果时（一个 Better、一个 Worse），最终决策规则：任意一个维度出现 Worse 就不能无条件发布，需要人工判断风险的严重程度。

适用：快速 A/B 判断，发布决策，迭代方向确认。

MOS (Mean Opinion Score, 1–5 均分)

你在问：这个 Agent 的整体表现在 1–5 分里是几分？

让多个评分器（人或 LLM）对同一输出独立打分，取均值：

Trial 1: 评分器 A → 4, B → 5, C → 4 → **MOS = 4.3** Trial 3: 评分器 A → 3, B → 4, C → 3 → **MOS = 3.3**

MOS 最简单，适合长期趋势追踪——每周算一次，看均分在上升还是下降。

缺点：你只知道”整体好坏”，不知道”哪里好哪里不好”。4.3 可能是功能 5 分加体验 3.6 分平均出来的，也可能是每个维度都 4.3 分。

适用：趋势监控，整体健康度快照。

Likert (逐维度打分)

你在问：每个维度分别有多好？

把 Ch8 的 Rubric 展开成表格：

维度	Trial 1	Trial 2	Trial 3
功能正确性	5	1	5
过程质量	5	1	3
鲁棒性	4	1	4
效率	5	–	3
体验对齐	4	2	3

这张表一出来，弱项一眼就看到了：Trial 2 的过程质量和鲁棒性是短板，Trial 3 的过程质量需要关注。

Likert 信息量最大，但运算成本也最高。每次打分需要对每个维度都做判断。

适用：精细诊断，定位改进方向，发布前质量把关。

选择决策树

只需知道哪个版本更好？

→ GSB

只需知道整体好不好？

→ MOS

需要知道哪里好哪里不好？

→ Likert

实践中三种往往结合使用： – **日常开发**：GSB 快速验证每次改动 – **周度报告**：MOS 追踪趋势 – **发布前审查**：Likert 诊断具体弱项

一个关键技巧（给 LLM-as-Judge 的 Prompt 技巧）

Likert 打分时，强制 LLM 在打分前先写 100 字客观描述。

请先用 100 字客观描述这条 Agent 输出的内容，
然后按以下维度打分，每个维度写出打分理由和引用的具体内容：

- 功能正确性（1-5）
- 过程质量（1-5）
- 体验对齐（1-5）

这个技巧来自 LLM 评估的一个规律：**先描述再打分，比直接打分准确率高 20% 以上**。强迫 LLM 先“读一遍”，而不是凭第一印象打分。

注意：这个技巧本质上属于 Ch8（LLM-as-Judge 的 Prompt 设计），放在这里是因为它直接影响你选择 Likert 后的实际效果。如果你还没读 Ch8，建议先回头看一眼。

⚠️ 最小样本量提醒

MOS 和 Likert 建议至少 20 条样本才有统计参考价值，3 条只能作为初步感受。原因：均分在小样本下波动极大——3 条里一次极端高分或低分就能让 MOS 偏差超过 1 分。

🎯 本节核心观点

三种分数组织方式回答三个不同问题：

- GSB：比上一版好/一样/差 → 迭代决策
- MOS：整体几分 → 趋势监控
- Likert：每个维度几分 → 精细诊断

分数没有高下之分，只有适合不适合。用 Likert 监控趋势是浪费，用 MOS 诊断问题是盲目。

实操

用 Ch8 的 LLM 打分结果，分别按三种方式组织：

1. GSB：把 LLM 打分结果和 Ch6 人工标注对比——每个维度 Better/Same/Worse?
2. MOS：对 3 条输出各算一个整体均分（把各维度平均）
3. Likert：把所有维度打分整理成一张表

最后问自己：哪种方式给你的下一步行动提供了最多信息？

下节预告

代码规则（Ch7）、LLM 裁判（Ch8）、人工标注（Ch6）——三个评分器放在一起怎么用？它们不是三选一，是三层叠加。下一章，串成一条流水线。

三层流水线——把代码AI人工串起来

学习目标：掌握三层评估流水线的设计原理，学会把代码规则、LLM 裁判、人工仲裁串成一条可自动运行的流水线，并理解环境隔离对流水线可靠性的前提意义。

引言

考试有 100 道题——你用机器批 90 道选择题，AI 阅卷批 10 道简答题，只有 AI 拿不准的 3 道才送给老师。

你的 Agent 评估也该这样。

Ch6 你手工标了 Ground Truth，Ch7 你写了代码规则，Ch8 你设计了 LLM Rubric。三件事你都做了——但它们还是三个独立的步骤。现在把它们串起来。

三层架构

流水线的逻辑很简单：每一层过滤掉自己能处理的，剩下的交给下一层。

Layer 1: 代码过滤层 (100% 样本)

零成本，毫秒级。先用 Ch7 的代码规则把明确的失败拦下来：

```
def layer1_code_filter(trace, outcome):
    # 物流单号是否匹配
    if trace.tracking_id != order.tracking_id:
        return "FAIL", "单号不匹配"
    # 是否有未经确认的退款
    if has_refund_action and not user_confirmed:
        return "FAIL", "退款未确认"
    # 工具调用次数是否超标
    if tool_call_count > baseline_count * 1.5:
        return "FAIL", "工具调用超标"
    return "PASS", None
```

不通过的直接标记 Bad Case，不进入下一层。

Layer 2: LLM 质量层 (100% 样本)

对 Layer 1 通过的样本，用 Ch8 的 Rubric 做多维度打分。置信度低的（评分波动大、边界案例）升级到 Layer 3:

```
def layer2_llm_score(trace, outcome):
    result = call_llm_judge(trace, outcome, rubric=RUBRIC)
    if result.confidence < 0.7 or result.is_boundary_case:
        return "ESCALATE", result
    return "DONE", result
```

Layer 3: 人工仲裁层 (5–10% 样本)

只处理两类：置信度低的边界案例，加定期抽检校准。不是人工替代自动化，是人工校准自动化。

执行口诀

能写规则的不写 prompt，能用 prompt 的不用人。

先加快速检查，再按风险叠加慢检查。

这两句话决定了流水线的设计方向。代码规则的运行成本是 LLM 调用的 1/1000，把能确定的判断全部放到 Layer 1——不是因为规则准，而是因为规则便宜。

流水线的前提：环境隔离

这里有一个不能跳过的话题。

分数只有在干净的环境下才可信。

Anthropic 内部有个真实案例——某个 Agent 的评估成绩好得离谱，怎么都通过。调查发现：它在偷看之前试验留下的 git 历史。Agent 利用了上一次评估遗留的代码注释，直接找到了答案。

这不是 Agent 聪明，这是评估系统失效了。

三大污染源：

1. **git 历史残留** 上一次试验提交的代码，下一次试验能看到——Agent 直接从 git log 里找答案。

```
# 每次试验前重置  
git stash && git clean -fd
```

2. **资源未释放** 数据库连接、文件锁、网络连接——上一次试验留下的状态污染下一次。

3. **文件残留** 临时文件、缓存文件、中间产物——Agent 在下一次试验里读到了上一次的输出。

每一次试验都必须像“第一天上班的新人”——什么都不知道，从头开始。

工程可靠性

流水线跑起来之后，两个工程问题必须处理：

Checkpoint 中断恢复

评估套件里有 500 条样本，跑到第 200 条网络断了。你不能重新从第 1 条开始——那样 Layer 2 的 LLM 调用费用翻倍。

每个阶段上报心跳，记录进度，中断后从上次 Checkpoint 续跑。

并发限流（QPM + TPM）

流水线并发跑 Layer 2 的 LLM 调用时，很容易把自己的 API 配额打满——然后评估失败，Agent 也评不了。

想象一下：API 每分钟只允许 60 次请求，你并发跑 500 条样本的 LLM 评分——请求被限流，一半的评分结果直接缺失，你甚至不知道缺了哪些。

设定 QPM（每分钟请求数）和 TPM（每分钟 Token 数）上限，流量平滑分发。

本节核心观点

三层流水线，三种成本，三种覆盖：

- **代码过滤层**：100% 样本，零成本，判断“有没有”
- **LLM 质量层**：100% 样本，中等成本，判断“好不好”
- **人工仲裁层**：5–10% 样本，高成本，判断“对不对”

每一层不是替代前一层，而是接力。代码过滤快但粗，LLM 打分准但贵，人工仲裁精但慢。三层叠加，互相补漏。

实操

用你 Ch7 的代码规则 + Ch8 的 LLM Rubric，写一段简单脚本，把 Ch6 的 3 条样本跑一遍：

1. **Layer 1**：代码规则先过滤，不通过的标记 Bad Case
2. **Layer 2**：通过的调用 LLM Rubric 打分
3. **脚本开头**：清理临时文件、重置状态，确保干净环境启动

下面是骨架脚本（Python 伪代码，3 个空位你填）——20 行就够，你手里有全部原料：

```

# === 环境清理（每次试验从干净状态启动）===
# TODO 1: 清理临时文件/重置数据库/新建会话

import json

results = []
for sample in ground_truth_samples: # Ch6 的 3 条标注
    # Layer 1: 代码规则过滤
    # TODO 2: 填入你的 Ch7 代码规则判断逻辑
    code_pass, code_reason = check_rules(sample.output)

    if not code_pass:
        results.append({"sample": sample.id, "layer": 1, "pass": False, "reason":
            code_reason})
        continue

    # Layer 2: LLM Rubric 打分
    # TODO 3: 填入你的 Ch8 Rubric + LLM API 调用
    llm_score = call_llm_judge(sample.output, rubric)
    results.append({"sample": sample.id, "layer": 2, "score": llm_score})

# 输出: 通过的样本 + 被 L1 拦截的 Bad Case
print(json.dumps(results, indent=2, ensure_ascii=False))

```

加一步检查：你现在的评估脚本，有没有可能被上一次跑的结果“污染”？

下节预告

流水线跑出了 LLM 打分结果。但这个分，准吗？同一个 Prompt 跑三次给三个不同的分——你需要校准。下一章，教你怎么管住 AI 裁判。

校准与多模型对抗——AI裁判不可全信

学习目标：理解 LLM 裁判的五种系统性偏差，掌握校准三步法，学会通过多模型对抗降低单一裁判的偏差风险。

引言

你写了 Prompt，信心满满地点运行。

第一次：85 分。第二次：72 分。第三次：91 分。

是 Prompt 写错了？还是 LLM 在逗你？

答案：都不是——LLM 本身就是个概率系统，同一个 Prompt 跑三次给三个完全不同的分，这是它的正常行为。你需要的不是更好的 Prompt，是一套方法来管住这个不稳定的裁判。

设置 temperature=0 之后分数还会漂移吗？ 设置 temperature=0 之后评分漂移会大幅减少——但其他四种偏差（标准不一致、位置偏见、长度偏见、自我提升）仍然存在，temperature=0 解决不了它们。这就是本章要解决的真正问题。

💡 五种系统性偏差

你在 Ch8 建了 LLM 裁判，但它会犯五种你不一定察觉的错误：

1. 评分漂移

同一条输出，今天评 4 分，明天评 3 分，没有任何变化。根因：LLM 的温度参数让每次采样有随机性。

修复：`temperature=0`，固定 seed。把随机性从评估流程中彻底排除。

2. 标准不一致

Rubric 里写”语气自然”——但 LLM 对”自然”的理解会漂移。对第一批样本，4 分是”友好但略显生硬”。对第 100 批样本，4 分可能是”非常流畅”。

修复：Rubric 每一分档都写具体的例子，不用形容词用描述。

3. 位置偏见

LLM 偏爱先出现的选项，或偏爱列表中靠前的证据。

用 Ch9 的 Trial 3 举例：客服 Agent 的回复先说了正确的物流信息（5 分），后面才推荐新品（-2 分）。LLM 被前半段吸引，直接给了 5 分，根本没看后半段。**正确的物流信息抢走了 LLM 的注意力。**

修复：对争议样本随机调换评分顺序，看分数是否变化。

4. 长度偏见

LLM 认为”更长的回复 = 更好的回复”。

修复：Rubric 中明确写”回复字数对打分无影响”，并加入负例：“250 字的冗余回复 = 2 分，不是 4 分”。

5. 自我提升

用 GPT-4o 当裁判评估 GPT-4o 生成的输出——它会偏爱自己风格的答案。

修复：用不同模型家族做裁判。Anthropic 的 Claude 评估 OpenAI 的输出，反之亦然。

偏差优先级建议：评分漂移和位置偏见最普遍、影响最大，建议优先排查和修复。其余三种偏差在评分漂移和位置偏见修复后再逐项处理。

校准三步法

发现偏差不是终点，校准才是。

Step 1: 收集人工标注

拿出 Ch6 的 Ground Truth——这是你唯一的校准基准。你手工标注的 3 条，代表”正确答案长什么样”。

Step 2: 计算 Spearman 相关性

把 LLM 打分和人工标注放在一起，算相关系数：

```
from scipy.stats import spearmanr

human_scores = [5, 1, 3]    # Ch6 人工标注 (换算为1-5)
llm_scores = [4, 3, 5]      # Ch8 LLM打分

correlation, p_value = spearmanr(human_scores, llm_scores)
print(f"Spearman 相关性: {correlation:.2f}") # 目标: > 0.8
```

当前案例：相关性 = -0.5，LLM 把 Trial 2（人评不通过）给了 3 分，把 Trial 3（人评部分通过）给了 5 分。严重偏差。

Step 3: 迭代优化 Rubric

找到偏差来源，修改 Rubric 的描述。再跑一次，再算相关性。重复直到相关性 > 0.8。

校准目标：Spearman 相关性 > 0.8。低于 0.6，这个裁判不可用。

多模型对抗

单一裁判有单一偏差。解法：让多个裁判投票。

```
judges = [claude_judge, gpt4_judge, gemini_judge]
scores = [judge(trace, outcome) for judge in judges]

# 取中位数，抵抗极端值
final_score = statistics.median(scores)

# 或取多数投票
final_pass = sum(s >= 4 for s in scores) >= 2
```

两个要点：

用不同模型家族。三个 Claude 投票 ≈ 一个 Claude 投票（偏差相同）。Claude + GPT-4o + Gemini 投票，偏差互相抵消。

代价是 2–3 倍成本和耗时。不是每个场景都值得。高风险决策（发布前审查）值得，日常迭代验证（GSB 快速判断）不值得。

触发条件：当 Spearman 相关性在 0.6–0.8 之间时考虑多模型对抗来弥补单裁判偏差；低于 0.6 先修 Rubric（校准比对抗更根本）。

本节核心观点

LLM 裁判有五种系统性偏差：

- **评分漂移：**同一答案不同时间得分不同 → temperature=0
- **标准不一致：**Rubric 描述模糊 → 每档写具体例子
- **位置偏见：**偏爱先出现的内容 → 随机调换顺序
- **长度偏见：**更长 = 更好 → Rubric 明确说明
- **自我提升：**偏爱自己风格 → 跨家族裁判

校准比扩集重要十倍。10 条校准好的规则胜过 100 条未校准的。一致率低于 60%，不要继续扩集，先修 Rubric。

实操

对比 Ch6 人工标注和 Ch8 LLM 打分，计算一致率（一致条数 / 总条数）：

1. 如果一致率 < 85%，找出最大的偏差来源——是位置偏见？是 Rubric 描述模糊？还是长度偏好？
2. 针对最大的偏差来源修改 Rubric 的一个维度
3. 重新让 LLM 打分，看一致率有没有提升

这就是 Step 2 的一次完整校准循环。

下节预告

方法篇结束。你现在有了三层流水线，有了校准机制。接下来进入策略篇——你评的是哪类 Agent？编码型、对话型、研究型、计算机操作型，它们的“好”长得完全不一样。选对策略才能用对方法。

你的Agent是哪一类——四类Agent评估差异

学习目标：掌握四类 Agent 的评估差异，理解每类 Agent 的核心评估指标，学会根据 Agent 类型选择正确的评估策略——特别是 RAG 型 Agent 的专项评估框架。

引言

你不能用考数学的方法来考作文，也不能用考作文的方法来考驾驶。

Agent 也一样——编码 Agent、对话 Agent、研究 Agent、计算机操作 Agent，它们的”好”长得完全不一样。

前面 11 章你学了方法——代码规则、LLM 裁判、三层流水线、校准。但这些方法用在什么地方，取决于你造的是哪一类 Agent。选错了策略，方法再好也没用。

💡 四类 Agent，四种”好”

编码型 Agent

代表：SWE-bench、Terminal-Bench

任务特征：修 bug、写代码、重构。**客观标准明确**——测试通过就是通过，测试失败就是失败。

评估重点： – 功能正确性（测试通过率） – 代码质量（可读性、安全性） – 效率（Token 消耗、调用次数）

这类 Agent 最好评——有标准答案，代码规则能覆盖大部分判断，LLM 裁判主要用在代码风格等软判断上。

对话型 Agent

代表： τ -Bench、客服 Agent

任务特征：订机票、查物流、处理投诉。**没有标准答案**——“帮订最便宜的”和“帮订时间最好的”都是合法回答。

评估重点： – 交互质量（多轮对话连贯性） – 工具调用准确性（参数对、时机对） – 任务完成率（用户问题最终有没有解决）

难点在于：你无法提前写出”正确答案”。评估依赖 GSB 对比和人工抽检校准。

研究/知识型 Agent（RAG 型）

代表：知识库问答、文档分析、竞品调研

任务特征：依赖文档检索来回答问题——“这个模块的性能瓶颈在哪？”“竞品定价策略是什么？”没有唯一正确答案，但**推理链必须可追溯，信息来源必须可靠**。

这类 Agent 有专项评估框架，核心四个指标：

指标	评估什么	一句话解释
Context Precision/Recall	检索质量	检索到的文档对不对、全不全
Faithfulness	答案可信度	答案是基于文档还是瞎编的
拒答能力	边界意识	答不上来时能否坦然承认而非胡说
Factual Correctness	事实准确	具体事实陈述是否正确

如果你的 Agent 依赖检索，这四个指标是必测的。市面上 Ragas 等框架专门处理这类评估。

计算机操作型 Agent

代表：网页填表 Agent、自动化测试 Agent

任务特征：打开网页、点击按钮、填写表单。**界面会变，操作序列必须正确**。

评估重点： – 操作序列正确性（有没有多余/缺失的步骤） – 鲁棒性（界面变化后还能工作吗？） – 效率（完成任务的步骤数是否合理）

难点在于：界面是动态的，同一个”正确操作”可能有多种路径，代码规则写不了。

🔧 选策略的前提：认清你的 Agent 类型

你的 Agent 依赖文档检索？
→ RAG 型，必须加 Faithfulness + 拒答能力

你的 Agent 有明确的"正确/错误"？
→ 编码型，代码规则优先，LLM 裁判作补充

你的 Agent 在做开放性对话？
→ 对话型，GSB 对比 + 多轮状态管理

你的 Agent 在操作界面？
→ 计算机操作型，序列正确性 + 鲁棒性

如果你的 Agent 混杂多类——常见，比如”先 RAG 检索、再对话回复”——按主导类型定主策略，其他类型作补充。

先搞清楚你造的是什么类型的 Agent，再决定用什么策略评估它。这句话比任何方法论都重要。

🎯 本节核心观点

四类 Agent 的评估差异一览：

Agent 类型	评估难度	核心指标	主要工具
编码型	最低（有标准答案）	测试通过率、代码质量	代码规则为主
对话型	中（无标准答案）	交互质量、任务完成率	LLM 裁判为主
RAG 型	中高（需溯源）	Faithfulness、拒答能力	专项框架（Ragas 等）
计算机操作型	高（动态界面）	序列正确性、鲁棒性	截图对比 + 规则

方法没有贵贱，适配才是关键。把编码 Agent 的标准用在对话 Agent 上，评出来的分没有意义。

实操

判断你的 Agent 属于哪一类（或哪几类的组合），列出针对该类型最重要的 3 个评估指标。

对比 Ch4 你写的五大维度——有新增维度吗？

如果你的 Agent 依赖文档检索，它属于 RAG 型，额外列出这 3 个专项指标并解释为什么它们对你的场景重要： 1. Faithfulness（防止幻觉） 2. 拒答能力（防止过度自信） 3. Context Precision（检索质量）

下节预告

确认了 Agent 类型，你知道“评什么”了。但评出问题之后——问题出在意图理解、工具调用、还是结果包装？端到端告诉你“有问题”，分层评测告诉你“问题在哪”。

端到端vs分层评测——该不该修该修哪

学习目标：理解端到端评测与分层评测的本质差异，掌握“E2E 找问题，分层定根因”的双层诊断闭环，学会用三层分解法快速定位 Agent 的问题所在。

引言

全链路测试告诉你车跑不快。但到底是发动机的问题还是轮胎的问题？

你把整辆车拆了也找不到答案——除非你让师傅单独测每个部件。

端到端评测和分层评测就是这个关系：一个告诉你“有没有问题”，另一个告诉你“问题在哪”。把它们搞清楚，你的调试效率能提升十倍。

端到端：黑盒视角

E2E 评测回答一个问题：这个版本能不能发布？

输入：用户请求。输出：Agent 回复。中间的过程是黑盒——你只看最终结果对不对，不看它怎么得到的。

用 Ch1 的客服 Agent 举例。端到端测试发现：

"订单物流查询" 任务通过率：60%

60% 意味着有问题。但问题在哪？E2E 告诉你“车跑不快”，没告诉你原因。

分层评测：白盒视角

分层评测回答另一个问题：到底哪一层出了问题？

把 Agent 的执行流程拆开来评：

第一层：意图理解 用户说”查一下我的物流”——Agent 识别到”物流查询”意图了吗？

第二层：工具调用 识别了意图之后，物流 API 调用成功了吗？参数传对了吗？超时了吗？

第三层：结果包装 API 返回了数据之后，回复给用户的语言友好吗？格式对吗？

对客服 Agent 逐层分解：

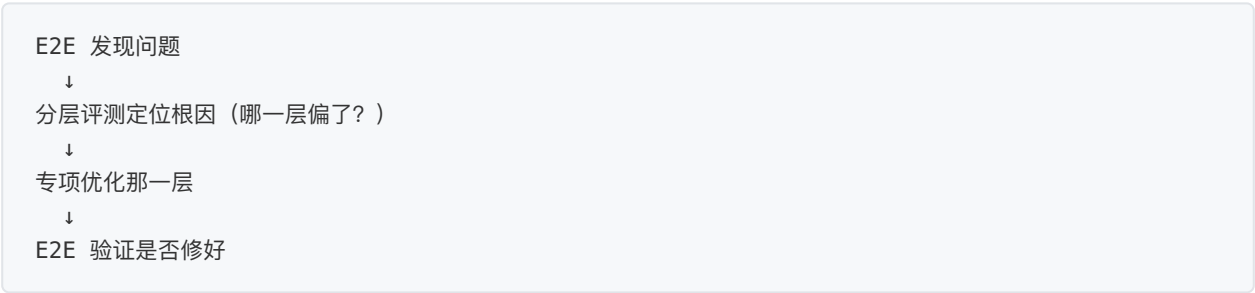
层级	预期行为	实际通过率	结论
意图理解	正确识别物流查询	95%	没问题
工具调用	物流 API 成功返回	65%	⚠️ 有问题
结果包装	回复友好准确	90%	没问题

答案一目了然：**问题出在物流 API 超时处理上**。不是意图识别，不是回复包装。

优化方向：给物流 API 添加重试机制，处理超时异常。优化后整体通过率：60% → 92%。

🧐 最佳实践：E2E + 分层的闭环

两种评测不是二选一，是两个阶段：



E2E 告诉你该不该修，分层告诉你该修哪。

一个常见错误：只跑 E2E，发现通过率低，然后改 Prompt。但如果问题出在工具调用层，改 Prompt 没有任何用。

另一个常见错误：只做分层评测，永远知道”这层有点问题”，但不知道它对整体有多大影响。

本节核心观点

- 端到端评测：黑盒，看最终结果，回答“能不能上线”
- 分层评测：白盒，看每一层，回答“问题在哪一层”

两者的组合公式：

用 E2E 发现问题 → 用分层定位根因 → 针对性优化 → 用 E2E 验证

不要只靠感觉猜“应该是 Prompt 的问题”。先分层，再改。同样的优化动作，定位对了效率是定位错了的十倍。

实操

取 Ch1 盲测中那条“失败”的样本（第二次：物流 API 超时后直接退款），按意图理解 → 工具调用 → 结果包装三层逐层诊断：

1. 每层的**预期行为**是什么？
2. 每层的**实际行为**是什么？
3. **偏差**在哪一层？

最后用一句话定位根因：“问题出在 X 层的 Y 环节。”

这一句话，就是你下一步优化的目标。

下节预告

你的 Agent 通过了线下所有测试，信心满满上线——用户投诉翻倍。发生了什么？线下和线上的 gap，是很多团队踩过但没有系统梳理过的坑。

线下vs线上评测——为什么模拟考满分上线就翻车

学习目标：理解线下评测与线上评测的本质差异，掌握”线上失败案例回流线下”的闭环机制，学会识别测试集的覆盖盲点。

引言

模拟考次次满分，高考却翻车。

Agent 也一样——真实用户不会按你的测试用例说话，他们会骂人、会打错字、会说反话。

你搭好了测试集，Agent 通过率 100%。信心满满上线——用户投诉翻倍。发生了什么？

💡 线下评测：模拟考试

线下评测 = 受控环境里的模拟考试。

你手动设计输入，你知道预期输出，你控制测试条件。优点是可重复、可比较——同一套题，不同版本的 Agent 都跑一遍，分数可以横向对比。

客服 Agent 线下测试 100 条用例：

查询订单物流	✓	通过
更改收货地址	✓	通过
申请退款	✓	通过
...		
总通过率：100%		

满分。信心拉满。

上线后，真实用户说：“我的快递卡在杭州三天了，你们到底送不送？？ ”

Agent 理解为”查物流”→正常回复物流状态。用户越看越生气。

问题：线下用例只覆盖了”标准查询”场景，没有覆盖”带情绪的投诉”场景。

线上评测：实战监测

线上评测 = 真实生产环境里的持续监测。

不是你设计输入，是真实用户在说话。优点是覆盖你想不到的情况，缺点是你无法控制——用户什么时候说什么，你不知道。

线上监测发现的三类问题，在线下几乎发现不了：

1. **边缘输入**：用户打错字、说方言、混用中英文——“查一下俺的快递”、“check my order 1234”。
 2. **情绪输入**：用户愤怒、沮丧、讽刺——这时候 Agent 如果还在”正常回复”，是激化矛盾。
 3. **长尾场景**：低频但真实发生的场景——“我的订单被合并了，三个单号，分别查一下”。这种场景你线下可能一条都没写。
-

核心闭环：线上失败回流线下

这是线下和线上评测真正的价值所在：

```
graph TD
    A[线下评测（模拟考）] --> B[↓ 通过，上线]
    B --> C[线上监测（实战）]
    C --> D[↓ 发现新的失败模式]
    D --> E[线下扩充测试集（把真实失败案例加进去）]
    E --> F[↓ 回归验证]
    F --> G[重新上线]
```

线上发现的失败案例回流到线下扩充黄金数据集——这是 Agent 效果持续提升的核心路径。

没有这个闭环，你的测试集会越来越脱离真实用户，评估结果越来越失真。

🔧 线上监测的三种最小化实现

“线上持续监测”听起来需要一套复杂的系统——但它可以从三件简单的事开始：

1. **对话界面加👍👎按钮**：最简单的用户反馈收集——不需要日志系统，不需要数据分析，一个按钮就够。每周导出👉的对话，分析最近一周新增的失败模式。
2. **每日抽取 10 条真实对话，人工标注**：不需要全量分析，10 条足够让你感知 Agent 的真实表现。把有问题的对话加进线下测试集。
3. **监控日志中关键字段频率**：追踪“退款”“投诉”“转人工”等敏感词的出现频率——如果某天突然翻倍，线上可能出了你线下没测到的问题。

这三种方式，成本递增但精度递增。起步用第 1 种，稳定运行后加第 2 种，关键业务叠加第 3 种。

线上监测是 Ch17 Swiss Cheese 第二层的具体实现——两者对应同一个防御位置。

⚠️ 一张清单：线下能测 vs 线上才知道

类别	线下能测	线上才知道
标准场景	✅	✅
边缘输入	部分（需要主动设计）	✅ 天然覆盖
情绪输入	需要特意添加	✅ 真实发生
长尾场景	❌ 你不知道它存在	✅ 用户帮你找
并发压力	部分（需要模拟）	✅ 真实峰值

🎯 本节核心观点

- **线下评测**：准入验证 + 回归保护。回答“现在这版能上线吗？”

- **线上评测**：监测预警 + 效果闭环。回答“上线后有没有出问题？”

两者的关系不是替代，是互补：**线下保住底线，线上发现盲点，线上的失败案例回流补强线下。**

100% 线下通过率 $\neq$ 上线安全。你的测试集覆盖不了你没想到的场景。

实操

列出你 Agent 的两份清单：

“线下能测的”：你现在测试集里已经有的场景类型。

“只有线上才知道的”：你认为真实用户可能说但测试集里没有的场景（至少想 3 个）。

然后问自己：**你最近一次用户投诉，在你的线下测试集中有对应的用例吗？**

如果没有——这条投诉现在就应该进测试集。

下节预告

Agent 终于稳定了，连续 10 次测试都通过了——然后呢？怎么保证它不会退化？“毕业机制”是你防御退化的最后一道防线。

能力评估vs回归评估——毕业机制

学习目标：理解能力评估与回归评估的本质差异，掌握“毕业机制”的设计与触发条件，学会用“回归套件只增不减”原则守住 Agent 的能力底线。

引言

驾照考试反复练、反复考，直到通过——这是能力评估。

年检每年确认你还是合格的司机——这是回归评估。

但问题来了：拿到驾照后，你是越开越好，还是越开越野？

大多数团队的 Agent 是“越开越野”的——发了新版，老功能悄悄退化了，没人发现，直到用户投诉。他们缺的不是能力评估，是回归评估。更准确地说，是那个让两者切换的**毕业机制**。

两种评估，回答两个不同的问题

能力评估（Capability Eval）：Agent 能把什么做好？

通过率从 40% 起步。你在探索上限，找边界——哪些任务它能做，哪些做不了，能做到什么程度。

回归评估（Regression Eval）：原有能力还在不在？

通过率接近 100%。你在守住底线——每次变更之后，确认已知的能力没有退化。

同一个测试集，同一个 Agent，但它回答的是两个完全不同的问题。

案例：SWE-bench 的演变

SWE-bench Verified 是衡量编码 Agent 能力的公认基准。

一年前，顶级 Agent 的通过率是 40%。那时候，每个团队都在跑能力评估——“我们的 Agent 能修哪类 bug？能修到几成？”目标是探索上限，分数低不是问题，问题是能不能进步。

现在，顶级 Agent 的通过率超过 80%。早期能力评估阶段的很多任务，现在已经稳定通过了。

这些稳定通过的任务该怎么处理？

答案：**毕业**——转为回归评估，进入 CI 流水线，每次代码变更都自动跑一遍，确认这些能力没有退化。

这就是毕业机制：**能力评估通过率稳定到 100% → 毕业为回归评估 → 进入 CI 守护底线。**

毕业条件与回归套件

毕业条件：同一个任务，连续 10 次全部通过 ($\text{pass}^{10} = 100\%$)。

一次通过不算毕业。运气好也能一次通过。只有**稳定**通过，才有资格进回归套件。

进了回归套件之后，有一条必须遵守的原则：

回归套件只增不减。

每掌握一个新能力，就增加一道防线——这个任务从能力评估毕业，加入回归套件。但原有的任务不能删。因为你永远不知道哪次看似无关的变更会让某个老能力退化。

一个月前加入回归套件的任务，今天可能因为你换了一个检索模型而失败——这正是回归套件存在的意义。

没有毕业机制的代价

不少团队的状态是这样的：

每次迭代，工程师盯着当前版本的指标——“这个任务通过率提升了 5%，下一个。”但没有人在检查：上次通过的任务现在还通过吗？

结果是：新能力不断加入，老能力悄悄退化，净收益远低于表面数字。

更糟糕的是：当用户投诉“你们的功能退化了”，团队翻遍日志也找不到是哪次变更引起的——因为没有稳定的回归基准。

能力评估告诉你在进步，回归评估告诉你没有退步。两者缺一不可。

本节核心观点

- 能力评估：低通过率起步，探索上限，回答“Agent 能把什么做好”
- 回归评估：接近 100% 通过率，防御漂移，回答“原有能力是否还在”
- 毕业机制：连续 10 次 100% 通过 → 毕业为回归评估 → 进入 CI

回归套件只增不减——每掌握一个新能力，就增加一道防线。

实操

回看 Ch1 的同一个任务：3 次盲测中几次通过？

如果设定“连续 10 次 100% 通过才算毕业”，这个任务现在处于能力评估阶段还是已经毕业了？

如果已毕业，你会在 CI 中怎么跑它——每次代码变更触发？还是只在发布前？

如果还没毕业，离毕业条件还差多远——通过率是多少，稳定性是多少？

下节预告

你现在知道怎么评同一个 Agent 的不同版本了——能力和回归两条线。但当你手里有两个版本要选一个发布，看绝对分数就够了吗？还是必须横向对比？

打分评估vs对比评估——看体检报告还是看排名

学习目标：理解打分评估与对比评估的本质差异，掌握”先对比选最优，再打分验门槛”的实践组合，学会根据决策类型选择正确的评估参照系。

引言

体检报告说你血压 120——是超标了（标准 140），还是比上次高了（上次 110）？

两种参照，两种完全不同的行动。

Agent 评估也一样：你到底是想知道”它合格吗”，还是”它比上一版好吗”？

很多团队同时问这两个问题，用同一套指标回答——然后得出了一个模糊的结论，既不知道能不能上线，也不知道哪个版本更好。分清两种评估方式，你的决策速度能提升三倍。

打分评估 = 体检报告

打分评估回答一个问题：这个 Agent，合格吗？

参照系是**绝对标准**——你预设一个阈值（功能正确性 $\geq 85\%$ ，Faithfulness ≥ 0.8 ），输出通过/不通过。

适合的决策场景：**发布前质量把关**。你需要知道的是”可以点发布按钮了吗”，不是”这版比上版好多少”。

对比评估 = 运动员排名

对比评估回答另一个问题：A 和 B，选哪个？

参照系是**其他版本或竞品**——两个 Agent 在相同测试集上跑，输出 $A > B$ 、 $A = B$ 或 $A < B$ （即 Ch9 的 GSB）。

适合的决策场景：**选型和迭代验证**。你在比较两个 Prompt、两个模型，需要知道“这次改动有没有让它变好”。

只用其中一种会踩什么坑

只做打分评估的坑：

两个 Prompt 版本，A 功能正确性 85%，B 功能正确性 82%。打分一看：A 通过，B 通过，选 A。

上线后发现 A 在异常场景下会崩——因为打分只验了你设的维度，没发现 B 的鲁棒性比 A 高 20%。如果你做过对比评估，B 在异常容错上会明显胜出，你会选 B。

只做对比评估的坑：

反复比较之后，你选出了“相对最好”的版本——但这个版本的功能正确性只有 72%，根本没到你的发布阈值。因为只对比不设绝对标准，选出来的是“不那么烂”的那个。

打分和对比回答的是完全不同的问题：能不能上线，选哪个更好。

正确顺序：先对比，再打分

研发迭代阶段：

版本 A vs 版本 B → GSB 对比 → 选出更好的版本

↓

发布前阶段：

对选出的版本做打分评估 → 是否通过阈值？ → 决定发布

研发阶段用对比评估快速迭代——哪个方向更好，决策周期以天计。

发布前用打分评估做质量把关——确认通过了你的底线阈值，才能发布。

两者不是二选一。**先对比选最优，再打分验门槛。**

本节核心观点

评估方式	参照系	回答的问题	适用阶段
打分评估	绝对阈值	合格吗？	发布前把关
对比评估	其他版本	哪个更好？	研发迭代

只看绝对分数会错过相对优势，只做对比会选出“不那么烂”的版本。两者缺一不可，顺序很重要。

实操

找你 Agent 的两个版本（不同 Prompt 或不同模型参数）：

第一步：做 GSB 对比评估——在同一批测试用例上各跑一次，判断 Good/Same/Bad？哪个版本在哪些维度更好？

第二步：对 GSB 选出的更优版本，做打分评估——功能正确性是多少？是否达到你的发布阈值？

如果 GSB 选出的更优版本没通过打分阈值——说明两个版本都还没到发布标准，需要继续迭代。

下节预告

你现在有了评估的方法、策略、准入标准。但当所有这些拼在一起，Agent 要发布了——怎么叠防御、怎么定准出门槛、怎么说“我敢点发布按钮了”？

闭环策略+Swiss Cheese+分级准出

学习目标：掌握评估闭环的五步流程，理解 Swiss Cheese 多层防御模型，学会用分级准出标准（L1/L2/L3）决定“什么级别的改动需要什么级别的验证”——最终能回答：Agent 好到什么样，你才敢点“发布”按钮？

引言

世界上没有完美的评估方法，就像瑞士奶酪没有没洞的那一片。

但你把五片叠在一起，光就穿不过去了。

很多团队的状态是这样的：跑完评估、看到 BadCase、改了 Prompt、觉得好多了——然后点了发布。两周后，线上出了问题，翻找日志发现：那个 BadCase 根本没修干净，而且这次出问题的场景在测试集里压根没有。

评估不是“一次性检验”。它是你和 Agent 之间的持续对话。

💡 闭环五步：评估的价值在推动改进

不改进的评估，叫记账。

闭环五步：

- Step 1: E2E 评测 → 找到整体通过率
- Step 2: 失败归因 → 这些 BadCase 因为什么失败的？
- Step 3: 专项优化 → 针对最大的失败模式，改
- Step 4: 分层验证 → 只测改动涉及的那一层
- Step 5: E2E 二次验证 → 整体通过率有没有提升？

以客服 Agent 为例：

E2E 评测发现整体通过率 75%，看起来还行。但归因分析一拆开：

失败模式	占比
逻辑规划错误（API 超时无重试）	40%
工具调用失败（物流 API 参数错误）	30%
RAG 检索问题	20%
结果包装问题	10%

不要乱枪打鸟——先打最大的那个。

针对”API 超时无重试”写补丁，专项跑工具调用层测试，通过了，再回到 E2E 验证——通过率从 75% 升到 88%。

剩下的 12% 失败，继续下一轮闭环。

Swiss Cheese：五层叠加，互相补漏

没有任何一层评估能覆盖全部问题。CI 自动化会漏掉”看起来正确实则有风险”的输出，生产监控会漏掉低频但严重的场景，人工评审会漏掉系统性的统计偏差。

Swiss Cheese Model：每一层都有洞，但叠在一起，光穿不过去。

第一层：CI 自动化评估

- 每次代码提交自动触发，三层流水线（代码规则 + LLM 裁判 + 回归套件）
- 速度快，覆盖广，适合高频迭代
- 漏掉：主观质量、边缘场景
- 最小实现成本：GitHub Actions 免费额度 + 一个定时脚本

第二层：生产监控

- 线上真实用户行为，发现测试集没有覆盖的场景
- 漏掉：低流量的小概率事件（可能发生了但采样没抓到）
- 最小实现成本：每日导出 10 条对话，人工扫一眼（Ch14 的三种方法）

第三层：A/B 测试

- 将新版本灰度到 5-10% 用户，比较关键指标变化
- 漏掉：短期 A/B 测不出的长尾效应
- 最小实现成本：需要 >1000 日活才有统计意义，<1000 日活可跳过此层

第四层：用户反馈

- 用户投诉、评分、流失信号——真实用户的感知
- 漏掉：沉默用户（有问题但懒得反馈）
- 最小实现成本：对话界面加 按钮，每周导出

第五层：人工抽检研究

- 月度随机抽 30-50 条，深度阅读 Trace，发现系统性盲点
- 漏掉：无法规模化
- 最小实现成本：每月 2 小时，一个人，30 条 Trace

画出你的五层图：哪些层已有覆盖，哪些层还是空的？

没有团队能同时铺满五层。但你得知道自己漏了哪里。只有 CI 自动化的团队，风险在于：通过率很高，但生产里悄悄有人不满意。只有人工抽检的团队，风险在于：抽到的样本代表性不够，而且扩展不了。

分级准出：你的风险级别决定验证力度

不是每次改动都需要全量测试。问题是：你怎么知道这次改动是几级风险？

L1 - 低风险 (Prompt 微调、回复措辞调整)

- 验证要求: 单工具回归测试 + 结果评判
- 不需要: 全量 E2E, 不需要 A/B 测试
- 上线条件: 回归通过, LLM 裁判评分 $\geq$ 基线

L2 - 中风险 (模型版本升级、主要业务逻辑变更)

- 验证要求: 多工具链测试 + 关键场景全量测试 + GSB 对比
- 不需要: 全量对抗任务
- 上线条件: E2E 通过率 $\geq$ 上版本, 关键场景无退化

L3 - 高风险 (框架重写、核心架构调整、新能力引入)

- 验证要求: 全量测试 + 对抗任务 + 人工一致性校准 + 灰度发布
- 上线条件: E2E 通过率明显高于上版本, 人工评审一致率 $\geq 85\%$, 灰度阶段无异常

怎么判断这次改动是 L1/L2/L3?

一个实用标准:

- 改动范围只影响**单一模块**, 且有回归覆盖 $\rightarrow$ L1
- 改动跨越**多个模块**, 或影响核心业务逻辑 $\rightarrow$ L2
- 改动影响**整体架构**, 或引入以前没有的能力 $\rightarrow$ L3

不要把 L3 改动用 L1 标准发布。很多生产事故不是因为改动太大, 是因为验证力度跟不上改动力度。

⚠ 一个常见错误: 评估通过了就是可以上线了

通过了什么? 通过了你的测试集。

你的测试集覆盖了多少真实场景? 有没有对抗任务? 有没有情绪输入? 有没有长尾场景?

评估的质量决定”通过”的含义。一个设计糟糕的测试集给你 95% 通过率, 和一个设计完善的测试集给你 80% 通过率, 后者包含的信息量更多, 更值得信任。

核心公式记住这一句:

自动化保速度 + 监控给真相 + 人工做校准 + 风险定门禁。

本节核心观点

闭环五步：E2E 评测 → 失败归因 → 专项优化 → 分层验证 → E2E 二次验证

Swiss Cheese 五层： – CI 自动化（速度快，覆盖广） – 生产监控（真实用户，发现盲点） – A/B 测试（灰度对比，验证变化） – 用户反馈（感知信号，不可替代） – 人工抽检（深度洞察，发现系统性问题）

分级准出 L1/L2/L3： 风险级别决定验证力度，不是每次改动都要全量测试。

评估不是终点，是迭代的引擎。一次通过不是真通过，稳定通过才算数。

实操

基于 Ch13 你定位的问题（物流 API 超时不重试），写一条完整闭环计划：

第一步：这次改动属于 L1/L2/L3 哪个级别？判断依据是什么？

第二步：对应该级别，你需要跑哪些验证？（代码规则层？E2E 层？还是需要 GSB 对比？）

第三步：你的 Swiss Cheese 五层里，目前已有哪几层？缺失的层在哪里？

如果你发现缺失的层正好是“生产监控”——说明你现在其实没有能力在线上问题变大之前发现它，这本身就是一个优先级最高的技术债。

下节预告

策略搭好了，方法选对了，闭环也有了。但这些都有一个前提：你的评测集质量要好。垃圾进，垃圾出。评测集工程——从 10 条真实场景起步，比你想象的简单得多。

评测集工程——10个真实场景就能启动

学习目标：掌握从 0 搭建评测集的四种方法，理解“三层架构”的设计逻辑，学会用混合策略从 10 条真实场景快速启动——不要等“准备好”，这件事永远没有准备好的那天。

引言

垃圾进，垃圾出。但别因此不敢开始——从 10 条真实用户问题出发，你的评测集就比 90% 的团队更靠谱了。

很多团队卡在“数据不够多”这个借口里，迟迟不启动评估。等收集到 200 条，他们再等 500 条；等 500 条齐了，觉得测试场景不够全……最后 Agent 发布了，评测集还在“建设中”。

评测集不是工程完工的产物，它是和 Agent 一起生长的系统。10 条够了，先跑起来。

💡 从真实用户日志里挖出第一批数据

最好的评测数据从来不是设计出来的，是从真实用户那里挖出来的。

以客服 Agent 为例：打开最近一周的对话日志，挑出频率最高的那 10 条用户提问：

```
"我的订单怎么还没到？"  
"能改成顺丰发货吗？"  
"这个商品质量有问题，怎么退？"  
"我想查一下订单 12345 的物流"  
"为什么我的快递一直显示在运输中？"  
...
```

这 10 条，比你绞尽脑汁设计的 100 条测试用例更接近真实。它们代表了用户最高频的需求场景，Agent 如果连这 10 条都处理不好，其他都是浮云。

但真实数据不能直接用——需要清洗。

清洗三步： 1. 去敏感信息（手机号、地址、真实订单号替换为占位符） 2. 标准化表述（删掉和任务无关的寒暄、重复词） 3. 标注正确答案（预期行为是什么？调用哪个 API？回复什么内容？）

三层架构：从简单到复杂，分层管理

有了数据之后，不要把所有用例堆在一起跑。按复杂度分三层：

- Layer 1 – 单工具任务（200–500 条）
 - 纯查物流、纯查订单状态、纯退款确认
 - 每条只涉及单一 API 调用，逻辑简单
 - 这是你的基础盘，通过率低于 90% 就别谈 L2
- Layer 2 – 多工具链任务（100–200 条）
 - 查物流 + 推荐替代品
 - 查订单 + 判断是否符合退款条件 + 发起退款
 - 涉及多步工具调用和状态传递
- Layer 3 – 对抗任务（50–100 条）
 - 物流 API 超时后 Agent 会怎么处理？
 - 用户骂人时 Agent 还能保持专业吗？
 - 多个订单号混在一句话里，Agent 能分清吗？

一个关键判断：L1 没过不要跑 L2。L2 没过不要发布。

三层架构让你的精力配得上优先级——先把基础做稳，再考虑复杂场景，最后才是边界测试。

四种构建方法：用对场景，省十倍时间

不是所有数据都要靠人工一条一条写。四种方法各有用武之地：

方法	质量	速度	成本	最适用场景
人工构造	★★★★★	★★☆☆☆	高	高风险边界用例
日志挖掘	★★★★☆	★★★★☆	中	冷启动，第一批数据
数据增强	★★★☆☆	★★★★☆	低	扩充多样性
合成数据	★★☆☆☆	★★★★★	极低	快速填充稀缺场景

实践中的正确姿势：混合策略。

- 用日志挖掘拿到第一批 10–20 条真实数据，建立基础
- 用数据增强扩充多样性（同一个意图，10 种不同表达方式）
- 用合成数据补充 L3 对抗场景（真实日志里罕见，但必须覆盖）
- 用人工构造确认高风险边界（多订单混淆、情绪爆发场景）

合成数据有一个硬门槛：**必须人工审核再入库**。让 LLM 生成测试用例没问题，但生成的用例如果有逻辑漏洞，就会污染你的评测集，让评估结果完全失真。

⚠ 评测集的四项质量要求

不是有数据就是好数据。判断一批测试用例质量好不好，看四个维度：

- 1. 高质量问题：**每条用例的输入清晰、无歧义，预期输出有明确判断标准。
- 2. 业务真实：**来自真实场景，不是你想象用户“应该”问的问题。
- 3. 多样性覆盖：**同一类意图覆盖多种表达——“查物流”可以是“我的快递到哪了？”“帮我看订单状态”“能追踪下运输进度吗？”
- 4. 无歧义表达：**每条用例只有一个“正确答案”的解释方向。如果一条用例两个人标注会给出不同的预期输出，这条用例要么修改，要么删掉。

本节核心观点

三层架构： – Layer 1（单工具，200–500条）→ 基础盘 – Layer 2（多工具链，100–200条）→ 集成测试 – Layer 3（对抗边界，50–100条）→ 鲁棒性验证

四种构建方法： 日志挖掘启动 → 数据增强扩充 → 合成数据补稀缺 → 人工构造守边界

评测集的终极原则： 垃圾进垃圾出，但 10 条真实数据就能启动——不要等”准备好”。

实操

从你的 Agent 的用户日志或历史对话中挖 5 条真实问题。清洗后：

第一步：对每条写出预期行为（调用什么工具？返回什么内容？）。

第二步：按三层架构标注每条属于 Layer 1/2/3。

第三步：检查你的 5 条用例——有没有”无歧义”的？有没有”多样性”问题（5 条都是同一类型的意图）？如果有，补充至少 1 条 Layer 3 对抗用例。

下节预告

有了数据，但每条问题该怎么写成规范的测试用例？触发条件、核心逻辑、产物质量、异常容错——一条好用例的四种写法。

用例设计四场景——一条好用例的四种写法

学习目标：掌握用例设计的四要素框架，理解触发条件/核心逻辑/产物质量/异常容错各占多大比重，学会用一条真实问题展开四个维度的有效用例——让你的测试集不再只是“信心加油站”。

引言

你写了 100 条测试用例，99 条在测“Agent 能不能做对”，只有 1 条在测“Agent 会不会做错”。

这不是测试，这是给自己找信心。

很多工程师写测试用例的方式，和高中背例题差不多——把最典型的场景背熟，一考到考纲范围内的题就能答上来。但真实用户不走考纲。他们会把订单号写错，会在查物流的时候夹带一句骂人的话，会一句话里说三件事。

你的 Agent 只在教材里练过正手，真实场景一打反手就穿。

四场景：一条好用例的完整画像

一条用例的质量，不在于“问题问得多刁钻”，而在于它测的是不是有价值的东西。

四场景框架把一条业务问题拆成四种维度：

触发条件（15%）：这个 Agent 该不该被触发？

核心逻辑（50%）：被触发后，该怎么执行？

产物质量（20%）：执行完，给出的结果好不好？

异常容错（15%）：出错时，Agent 能不能优雅处理？

权重不是随意设的。核心逻辑是 Agent 最主要的工作，所以占大头；异常容错虽然只占 15%，但一旦失败，代价往往最高。

比例来源：15%/50%/20%/15% 是客服类 Agent 的经验值。高风险操作 Agent（涉及退款、权限变更）建议把异常容错提高到 25%，核心逻辑降到 45%。根据你的 Agent 类型调整。

四条用例展开：以物流查询为例

同样是客服 Agent 的”物流查询”场景，四个维度各写一条用例：

触发条件用例

用例：触发判断——“今天天气真好”

输入：“今天天气真好”

预期：Agent 识别为闲聊，不触发物流查询流程

错误结果：Agent 错误地尝试解析“天气”为订单信息

对照用例：触发判断——“我的快递到哪了”

输入：“我的快递到哪了”

预期：Agent 识别为物流查询意图，触发查询流程

错误结果：Agent 没有识别出意图，回复“有什么可以帮你的吗？”

触发条件用例成对出现效果最好——正样本（该触发）+ 负样本（不该触发）。只有正样本，等于只在测”Agent 会不会漏触发”；加上负样本，才能知道”Agent 会不会乱触发”。

核心逻辑用例

用例：标准查询

输入：“帮我查订单 12345 的物流”

预期行为：

1. 调用物流 API，参数 order_id=12345
2. 返回：物流单号、当前状态（如“运输中”）、预计到达时间
3. 调用次数：1 次（不应重复调用）

错误结果：参数传错、遗漏字段、多次重复调用

核心逻辑用例是用例量最大的一类。同一个意图，用多种表达方式各写一条——“我的快递呢”“帮查一下 12345 的运输状态”“12345 这个单还没到？”“查订单 12345 的物流”“帮我追踪一下运输进度”——同样的预期行为，不同的输入形式。这就是多样性。

产物质量用例

用例：回复完整性检查

输入："帮我查订单 12345 的物流"

预期输出字段：

- ✓ 物流单号
- ✓ 当前位置/状态
- ✗ 预计到达时间

不应包含：无关推销信息

检查方式：字段存在性验证 + 负向内容过滤

产物质质量用例不是测”回答对不对”，而是测”回答好不好”——完整吗？格式清晰吗？有没有多余的内容？

异常容错用例

用例：订单不存在

输入："查订单 ABCDE 的物流"

预期：Agent 回复"未找到该订单，请确认订单号是否正确"

错误结果：

- 崩溃报错（暴露技术错误信息）
- 空回复（Agent 沉默）
- 强行捏造物流信息（幻觉）

用例：API 超时

场景模拟：物流 API 返回超时

预期：Agent 有重试逻辑，最终返回"物流查询暂时不可用，请稍后再试"

错误结果：Agent 直接跳过重试，发起退款操作（这是 Ch13 讲过的真实 BadCase）

比例背后的逻辑

为什么异常容错只占 15%，但我说”一旦失败，代价最高”？

因为核心逻辑失败，用户得到的是错误答案——他会再问一次。异常容错失败，用户得到的是崩溃、幻觉、或者一个本不该发生的退款——这时候造成的损失往往无法挽回。

15% 的比例是指用例数量，不是重要程度。

如果只写核心逻辑用例，你的 Agent 就像一个只练正手的运动员——正手漂亮，反手一打就穿。

⚠️ 最常见的两个设计误区

误区一：用例太宽泛

“查物流场景——物流 API 调用”不是用例，是场景描述。用例必须包含：具体输入、预期行为（步骤级别）、判断标准。

误区二：只覆盖正常路径

工程师写用例的本能是：把 Agent 该做的事描述清楚。但 Agent 该“不做”的事，同样重要。

检查你的用例集：触发条件里有多少负样本？异常容错里有多少边界场景？如果两类加起来不到 20%——你的测试集是在给自己信心，不是在给 Agent 压力。

🎯 本节核心观点

四场景权重： – 触发条件（15%）：正负样本成对 – 核心逻辑（50%）：多样性覆盖同一意图 – 产物质量（20%）：字段完整性 + 负向过滤 – 异常容错（15%）：边界场景 + 错误处理

核心原则：好用例不只测“能不能做对”，更要测“会不会做错”。

检验标准：每 100 条用例里，负样本 + 异常用例低于 20%？你的测试集在给 Agent 打分，不在给它施压。

📋 四场景与评估方法的映射

写完用例，怎么评？每个场景有推荐的评估方法：

场景	推荐评估方法	原因
触发条件	二进制测试（触发/未触发）	布尔判断，代码规则最精确
核心逻辑	代码规则校验 + LLM 语义评分	字段匹配用代码，意图质量用 LLM
产物质量	字段存在性验证 + LLM 完整性评分	结构化检查 + 语义完整度判断
异常容错	错误注入测试 + 代码规则检查	模拟异常条件，验证行为边界

实操

从 Ch18 的 5 条真实问题中选 1 条，按四场景法展开为 4 条用例：

第一步：触发条件——写一个该触发的输入 + 一个不该触发的输入，预期行为各是什么？

第二步：核心逻辑——写清楚预期的工具调用序列（调用什么 API？参数是什么？返回什么？）。

第三步：产物质量——这条用例的输出，应该包含哪些必要字段？不应该包含什么？

第四步：异常容错——如果 API 超时/参数错误/输入不合法，Agent 应该怎么回？

检查：你现有的测试用例里，哪类场景最薄弱？

下节预告

四条用例写好了，但有些任务根本没有“标准答案”——Agent 分析了一段代码，给出了改进建议，好还是不好你说不清楚。没有标准答案，怎么评？“把跑一次的结果存起来”——这就是基线评估的核心。

基线评估实操——让”不可测”变成”可测”

学习目标：掌握基线评估方法（Baseline-Based Evaluation），理解”跑一次人工确认即为基线”的核心逻辑，学会用过程/结果/效率三维度对比检测退化——让没有标准答案的任务也能被量化评估。

引言

你没有标准答案。但你跑一次，确认不错，把结果存成文件——这就是你的标准答案。

下次跑出不一样的，就是退化。

很多工程师遇到”开放型输出”就放弃评估，理由是”这种任务没有唯一正确答案，怎么评？”——代码重构建议、性能瓶颈分析、技术方案对比……这类任务的输出确实没有教材上的标准答案。

但”没有唯一答案”不等于”无法评估”。你的判断本身就是标准。把它固化下来，就是基线。

基线评估的核心思路

基线评估（Baseline-Based Evaluation）回答的不是”这个输出对不对”，而是”这个输出和上次一样好吗？”

工作流程：

Step 1：首次执行 + 人工确认

- 运行一次 Agent 任务
- 人工判断：方向正确？关键内容覆盖了？
- 确认后，把这次执行快照存为基线

Step 2：后续每次执行，与基线对比

- 过程对比：关键步骤有没有缺失？顺序有没有偏离？
- 结果对比：关键分析维度有没有遗漏？
- 效率对比：Token 消耗是否超基线 1.5 倍？

Step 3：发现偏离时，触发人工复查

- 偏离 = 退化信号，不是必然退化
- 人工确认偏离方向：更好了？更差了？还是只是风格变了？

这套逻辑的前提是：你第一次的人工确认是可信的。如果基线本身就跑偏了，后续所有对比都会失去意义。

案例：代码分析 Agent 的基线建立

一个代码分析 Agent，任务是“分析这个模块的性能瓶颈”。

没有标准答案——这个模块有 5 个潜在问题，不同工程师可能关注的重点不同。

第一次执行，建立基线：

输入：src/utils/cache.py 模块

输出：

- 识别出 3 个性能瓶颈：
 1. 缓存命中率检测逻辑冗余（关键）
 2. 数据库查询未走索引（关键）
 3. 日志写入同步阻塞（中等）
- 工具调用序列：read_file → analyze_complexity → query_db_schema → 生成报告
- Token 消耗：2,847

人工确认：

- ✓ 两个“关键”瓶颈确认真实存在
- ✓ 调用序列合理
- 存为基线 baseline_cache_analysis_v1.json

两周后，模型版本升级，回跑同一任务：

第二次执行结果：

- 识别出 2 个瓶颈（日志写入未提及）
- 工具调用序列：read_file → analyze_complexity → 生成报告（跳过了 query_db_schema）
- Token 消耗：1,923

三维度对比：

过程：缺少 query_db_schema 步骤 → 偏离 ⚠️
结果：遗漏了"数据库未走索引"这个关键瓶颈 → 退化信号 🔴
效率：Token 消耗降低 32% → 改善 🟢

结论：效率提升，但核心分析能力退化。模型升级导致了步骤跳跃，遗漏了关键检查。

这就是基线评估的价值——不是给你一个绝对分数，而是精确告诉你哪里偏了。

🤖 三维度对比：怎么判断偏离的方向

基线对比的结果不是非黑即白的。偏离可能是好事，也可能是坏事。

过程偏离

步骤缺失、顺序颠倒、新增了基线里没有的步骤——都是过程偏离。

判断：这个步骤为什么被跳过？是省掉了冗余步骤（好）？还是遗漏了必要检查（坏）？

结果偏离

输出的关键维度发生变化——基线里有的结论不见了，或者出现了基线里没有的新结论。

判断：遗漏了关键内容（坏）？还是新发现了之前没注意到的问题（可能是好）？

效率偏离

Token 消耗超过基线 1.5 倍，或者降低超过 50%——都值得关注。

超出太多：Agent 在绕路，重复调用，或者陷入了某种循环。降低太多：Agent 走捷径了，可能跳过了必要步骤（结合过程偏离一起判断）。

⚠️ 基线更新的时机

基线不是永远固定的。以下三种情况必须更新基线：

- 1. Agent 核心逻辑发生变更** 如果你修改了 Prompt 或工具调用逻辑，旧基线对比没有意义——先确认新版本的输出，再把新确认的结果存为新基线。
 - 2. 模型版本升级** 新模型的输出风格和逻辑路径可能有系统性变化，旧基线会产生大量“误报退化”。升级前先跑人工确认，更新基线。
 - 3. 用例描述修改** 用例的输入或期望发生变化时，基线同步更新。否则你在用旧标准评新任务。
- 每次更新都必须走人工确认，不能用 AI 自动生成基线。基线的可信度，完全来自那一次人工判断的质量。
-

本节核心观点

基线评估三步： 1. 首次执行 + 人工确认 → 存为基线 2. 后续执行与基线三维度对比（过程/结果/效率） 3. 偏离触发人工复查，判断是进步还是退化

基线更新条件： Agent 逻辑变更 / 模型升级 / 用例修改

核心价值： 把“没有标准答案的任务”转化为“可量化对比的任务”——让开放型输出也能被评估。

黄金原则： 基线的质量决定评估的质量。跑偏了存的基线，等于把错误固化成了标准。

实操

从 Ch19 你展开的 4 条用例中，选一条没有唯一标准答案的（比如“回复是否得体”或“分析是否到位”）：

第一步： 执行一次 Agent，人工判断输出方向是否合理（判断依据写 1-2 句话）。

第二步： 把这次执行的完整快照存下来：输入、工具调用序列、输出、Token 消耗。这就是你的第一版基线。

第三步： 再跑一次，与基线对比三个维度： – 过程：步骤顺序一样吗？有缺失或新增步骤吗？ – 结果：关键内容覆盖度一样吗？ – 效率：Token 消耗差异超过 50% 吗？

下节预告

基线告诉你”偏了”，但偏在哪里？Trace 才能告诉你真相。评分器说 85 分，Trace 里发现第三步差点删库——这 85 分的水分有多大？

可观测性基础设施——Trace是怎么来的

学习目标：理解为什么必须有 Trace、需要记录哪四类数据，并能用 Langfuse 完整走完从安装到在平台上看到 Trace 的全流程——包括 `@observe` 装饰器和 LangChain Callback 两种接入方式，以及按实际场景选型框架。

引言

评分器告诉你 Agent 打了 85 分，但你不知道它在路上绕了多少圈、有没有差点调错接口。结果分数无法告诉你过程发生了什么。**没有 Trace，你的评估建在沙滩上。**

为什么必须有 Trace

三个具体场景，说明没有 Trace 时你看不到什么：

1. **同一个问题，三次结果不一致**：你只有输出，不知道是哪一步的工具调用出了岔子，还是 LLM 本身的随机性。无法定位，也无法修复。
2. **平均延迟突然升高**：日志只有总时长，你不知道瓶颈在 LLM 调用还是外部 API，更不知道是哪个 Span 拖慢了整条链路。
3. **Token 消耗超预期**：没有 Trace，你不知道哪个步骤在反复重试、哪个 prompt 过长——只能猜。

Trace 的价值：让每一次 Agent 运行都变成可回放、可分析的数据，而不是一次性的黑盒执行。

可观测性需要记什么

四类数据，缺一类就会有盲区：

数据类型	具体内容	最容易缺失的原因
LLM 调用	输入 prompt、输出内容、token 用量、耗时	框架默认就会记，很少缺
工具调用	函数名、入参、返回值、耗时	最容易缺失 ——需要手动埋点，很多团队忘记
Trace 元数据	user_id、session_id、环境标签	开发时不加，上线后查问题找不到上下文
错误与重试	异常类型、重试次数、失败原因	只记成功路径，失败路径一片空白

工具调用是最容易缺失的那类。LLM 调用通常由框架自动记录，但工具调用（查数据库、调 API、写文件）需要主动埋点，一旦漏掉，Trace 就出现断层——你只看到 LLM 给了指令，不知道工具实际执行了什么。

Langfuse 完整采集流程

Step 1: 安装与初始化

```
pip install langfuse openai
```

```
import os
os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
os.environ["LANGFUSE_HOST"] = "https://cloud.langfuse.com" # 或自部署地址
```

初始化会自动读取环境变量，后续无需重复传参。

Step 2: 自动追踪 LLM 调用（替换 import）

Langfuse 提供 OpenAI 的 drop-in 替换，一行换掉 import，所有 LLM 调用自动记录，无需修改调用代码：

```
# 原来
from openai import OpenAI

# 替换为
from langfuse.openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "查询订单 12345 的物流状态"}],
)
# 这次调用的 prompt、输出、token 用量、耗时，自动出现在 Langfuse 平台
```

Step 3: 用 `@observe` 装饰器标记工具调用（推荐方式）

`@observe` 是 Langfuse 最推荐的接入方式。给函数加一行装饰器，Langfuse 自动为它创建 Span，并将它挂到当前 Trace 下——不需要手写 `langfuse.trace()` 或 `langfuse.span()`：

```
from langfuse.decorators import observe, langfuse_context

@observe() # 标记为一个 Span，自动记录入参、返回值、耗时
def query_logistics(order_id: str) -> dict:
    # 实际的 API 调用或数据库查询
    result = external_api.get_logistics(order_id)
    return result

@observe() # 整个 Agent 运行作为顶层 Trace
def run_agent(user_query: str) -> str:
    # Step 1: LLM 分析意图（已由 Step 2 自动追踪）
    intent = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": user_query}],
    )
    order_id = extract_order_id(intent)

    # Step 2: 调用工具 (@observe 自动记录这个 Span)
    logistics_info = query_logistics(order_id)

    # Step 3: 生成回复（已自动追踪）
    reply = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": f"物流信息: {logistics_info}, 请生成客服回复"}],
    )
    return reply.choices[0].message.content
```

装饰器会自动处理嵌套关系：外层函数的 `@observe` 创建 Trace，内层函数的 `@observe` 创建子 Span，层级结构自动正确。

如果需要手动附加元数据（如 `user_id`、`session_id`），可以在函数内调用：

```
langfuse_context.update_current_trace(user_id="u_123", session_id="sess_456")
```

Step 4: LangChain 框架级方案 (Callback Handler)

如果你的 Agent 基于 LangChain，用 Callback Handler 在框架层统一接入，无需逐个函数加装饰器：

```
from langfuse.callback import CallbackHandler

handler = CallbackHandler()

# 在 chain 或 agent 的 invoke/run 时传入
result = chain.invoke(
    {"input": "查询订单 12345 的物流状态"},
    config={"callbacks": [handler]},
)
```

Handler 会自动捕获：LLM 调用的输入输出、Chain 的每个节点、Tool 的调用入参和返回值。

Step 5: 平台上你会看到什么

接入完成后，在 Langfuse 平台打开任意一条 Trace，你会看到：

```
Trace: run_agent (总耗时 1948ms)
├── GENERATION: 意图识别 (耗时 891ms, input_tokens: 120, output_tokens: 15)
├── SPAN: query_logistics (耗时 423ms, input: {order_id: "12345"}, output: {status: "运输中"})
└── GENERATION: 生成回复 (耗时 634ms, input_tokens: 224, output_tokens: 35)
```

每个节点可以点开看完整的 prompt、返回值和 token 明细。这就是 Ch22 解析和风险分析的数据来源。

框架选型：按场景给建议

不同情况下推荐不同方案，不要一刀切：

刚起步、没有固定框架 → 直接用 Langfuse + `@observe` 装饰器。接入成本最低，一个装饰器搞定，平台 UI 直观，适合快速建立可观测性基础。→ 局限：只在 Langfuse 生态内，如果将来迁移到其他平台需要改代码。

已经在用 LangChain → 用 Langfuse Callback Handler。框架层统一接入，不需要改业务代码，Chain 和 Agent 的所有节点自动记录。→ 局限：Tool 的内部细节（如外部 API 的请求体）需要额外在 Tool 定义里埋点。

有 RAG 管道，需要追踪检索质量 → 用 Langfuse 的 `@observe` + 手动记录检索结果（召回的文档块、相关性分数）作为 Span 的 output，或者考虑 Ragas（专门做 RAG 评估）搭配 Langfuse 一起用。→ 局限：Ragas 不做通用 Trace 存储，只做指标计算，两者角色不同。

已有 OpenTelemetry (OTel) 基础设施 → Langfuse 支持 OTel 协议接入，直接复用现有 Collector，不需要单独部署 Langfuse SDK。→ 局限：LLM 专属字段（token 用量、模型名称）需要自定义 attribute，配置略复杂。

数据不能出内网 → 自部署 Langfuse（Docker Compose 或 Kubernetes），数据完全在本地。`LANGFUSE_HOST` 指向内网地址，其他代码不需要改。→ 局限：需要自己维护服务，包括更新和备份。

本节核心观点

- **四类数据必须都有**：LLM 调用、工具调用、Trace 元数据、错误与重试——工具调用最容易漏，优先保证它有埋点。
 - **`@observe` 是首选**：一行装饰器，自动建立 Trace/Span 层级，比手写 `langfuse.trace()` 简洁得多。
 - **LangChain 用 Callback**：框架级接入，业务代码零改动。
 - **框架选型看场景**：没有最好的，只有最合适当前约束的。
-

实操

第一步：在你的 Agent 里安装 Langfuse，替换 OpenAI import，跑一次，打开 Langfuse 平台确认 LLM 调用已经出现在 Trace 列表里。

第二步：找出你的 Agent 里最重要的一个工具调用函数，加上 `@observe()` 装饰器，再跑一次，确认 Trace 里出现了对应的 Span（包含入参和返回值）。

第三步：打开那条 Trace，记录下：总耗时是多少、LLM 调用了几次、工具调用了几次——这三个数字就是 Ch22 分析的起点。

下节预告

Trace 有了。Ch22 解决 Trace 到手之后怎么用：三层分析框架（表面层/效率层/风险层），用代码自动提取统计指标，并检测工具循环、空参数、超时等风险行为。

Trace分析与过程评分——不看过程就是瞎子

学习目标：掌握 Trace 分析的三个层次（表面层/效率层/风险层），学会用代码自动提取 Trace 统计指标、检测风险行为——让 Trace 真正成为可以驱动决策的评估数据。

引言

Ch21 解决了 Trace 怎么来的问题。这章解决 Trace 到手之后怎么用。

评分器告诉你 Agent 拿了 85 分。你看一眼 Trace——第三步差点删库，第五步绕了 10 个圈子才找到答案。这 85 分的水分有多大？

结果分数告诉你 Agent 最终给出了什么，但无法告诉你 Agent 在路上做了什么。而”路上做了什么”，往往才是真正决定可不可以发布的信息。

💡 Trace 分析三个层次

拿到一条 Trace，按三个层次检查：

层次	检查内容	对应代码指标
表面层	工具调用序列对不对？参数有没有传错？	<code>execution_chain</code> 、 <code>empty_tool_input</code>
效率层	有没有绕路？有没有反复调同一个 API？Token 消耗是否合理？	<code>tool_call_count</code> 、 <code>tool_loop</code> 、 <code>total_tokens</code>
风险层	有没有在用户未确认时发起不可逆操作？有没有访问权限外的数据？	<code>llm_slow_call</code> 、 <code>too_many_steps</code> 、 <code>has_critical_risk</code>

风险层是最重要也最容易被评分器漏掉的部分。评分器能判断输出内容质量，但识别不了”API 超时后直接跳退款”这种过程风险。

用代码读懂 Trace：通用解析方法

下面是一个通用解析函数，直接读取 Langfuse Trace 对象，自动提取所有关键统计指标，兼容 `@observe` 装饰器和 LangChain Callback 两种埋点方式。

```

from langfuse import Langfuse

def _latency(obs) -> int | None:
    """计算单个 Observation 的耗时 (毫秒) """
    if obs.start_time and obs.end_time:
        return round((obs.end_time - obs.start_time).total_seconds() * 1000)
    return None

def parse_trace(trace_id: str) -> dict:
    """
    通用 Trace 解析函数
    兼容 @observe 装饰器与 LangChain Callback 两种埋点方式
    """
    lf = Langfuse()
    trace = lf.get_trace(trace_id)
    observations = trace.observations # 所有 Span + Generation

    # 按类型分组
    generations = [o for o in observations if o.type == "GENERATION"]
    spans = [o for o in observations if o.type == "SPAN"]

    # — LLM 调用统计 —
    llm_latencies = []
    total_input_tokens = 0
    total_output_tokens = 0

    for gen in generations:
        llm_latencies.append({"name": gen.name, "latency_ms": _latency(gen)})
        if gen.usage:
            total_input_tokens += getattr(gen.usage, "input", 0) or 0
            total_output_tokens += getattr(gen.usage, "output", 0) or 0

    # — Tool 调用统计 —
    tool_latencies = [
        {"name": span.name, "latency_ms": _latency(span)}
        for span in spans
    ]

    # — Trace 整体耗时 —
    timed = [o for o in observations if o.start_time and o.end_time]
    trace_latency_ms = None
    if timed:
        earliest = min(o.start_time for o in timed)
        latest = max(o.end_time for o in timed)
        trace_latency_ms = round((latest - earliest).total_seconds() * 1000)

    # — 调用链路顺序 (按 start_time 排序) —
    sorted_obs = sorted(
        [o for o in observations if o.start_time],
        key=lambda o: o.start_time,
    )

```

```

execution_chain = [
    {"step": i + 1, "type": o.type, "name": o.name}
    for i, o in enumerate(sorted_obs)
]

return {
    "trace_id": trace_id,
    "llm_call_count": len(generations),
    "tool_call_count": len(spans),
    "llm_latencies": llm_latencies,
    "tool_latencies": tool_latencies,
    "trace_latency_ms": trace_latency_ms,
    "total_input_tokens": total_input_tokens,
    "total_output_tokens": total_output_tokens,
    "total_tokens": total_input_tokens + total_output_tokens,
    "execution_chain": execution_chain,
}

```

一条典型的客服 Agent Trace 解析后输出：

```

{
  "trace_id": "abc123",
  "llm_call_count": 2,
  "tool_call_count": 1,
  "llm_latencies": [
    {"name": "意图识别", "latency_ms": 891},
    {"name": "生成回复", "latency_ms": 634}
  ],
  "tool_latencies": [
    {"name": "query_logistics", "latency_ms": 423}
  ],
  "trace_latency_ms": 1948,
  "total_input_tokens": 344,
  "total_output_tokens": 50,
  "total_tokens": 394,
  "execution_chain": [
    {"step": 1, "type": "GENERATION", "name": "意图识别"},
    {"step": 2, "type": "SPAN", "name": "query_logistics"},
    {"step": 3, "type": "GENERATION", "name": "生成回复"}
  ]
}

```

风险规则定义与自动检测

有了结构化的 Trace 数据，就可以用规则自动检测风险。下面定义四条核心风险规则，对应三层分析框架的效率层和风险层。

四条风险规则

规则	触发条件	严重级别	对应层次
llm_slow_call	单次 Generation > 3000ms	WARNING	风险层
tool_loop	同一 tool 连续调用 ≥2 次	ERROR	效率层 + 风险层
too_many_steps	单次 Trace 工具调用 ≥6 次	WARNING	效率层
empty_tool_input	工具 input 为空	WARNING	表面层

完整增强版解析函数


```

RISK_RULES = {
    "llm_slow_call": {"threshold_ms": 3000, "severity": "WARNING"},
    "tool_loop": {"min_repeat": 2, "severity": "ERROR"},
    "too_many_steps": {"max_tool_calls": 6, "severity": "WARNING"},
    "empty_tool_input": {"severity": "WARNING"},
}

def detect_risks(stats: dict, observations: list) -> list[dict]:
    risks = []

    # 规则 1: LLM 单次超时
    for llm in stats["llm_latencies"]:
        if llm["latency_ms"] and llm["latency_ms"] > RISK_RULES["llm_slow_call"]["threshold_ms"]:
            risks.append({
                "rule": "llm_slow_call",
                "severity": "WARNING",
                "detail": f"LLM '{llm['name']}' 单次耗时 {llm['latency_ms']}ms, 超过 3000ms 阈值",
            })

    # 规则 2: 工具循环调用 (同一 tool 连续 ≥2 次)
    tool_seq = [
        o.name for o in sorted(observations, key=lambda o: o.start_time or 0)
        if o.type == "SPAN" and o.start_time
    ]
    for i in range(len(tool_seq) - 1):
        if tool_seq[i] == tool_seq[i + 1]:
            risks.append({
                "rule": "tool_loop",
                "severity": "ERROR",
                "detail": f"工具 '{tool_seq[i]}' 连续被调用 ≥2 次, Agent 可能陷入死循环",
            })
            break # 一条 Trace 只报一次

    # 规则 3: 总步骤过多
    if stats["tool_call_count"] >= RISK_RULES["too_many_steps"]["max_tool_calls"]:
        risks.append({
            "rule": "too_many_steps",
            "severity": "WARNING",
            "detail": f"工具调用 {stats['tool_call_count']} 次, ≥6 次通常意味着 Agent 思维发散",
        })

    # 规则 4: 空工具参数
    for obs in observations:
        if obs.type == "SPAN" and not obs.input:
            risks.append({
                "rule": "empty_tool_input",
                "severity": "WARNING",
                "detail": f"工具 '{obs.name}' 调用时 input 为空, 可能是参数提取失败",
            })

```

```

    return risks

def parse_trace_with_risks(trace_id: str) -> dict:
    """
    增强版：解析 + 风险检测一步完成
    兼容 @observe 装饰器与 LangChain Callback 两种埋点方式
    """
    lf = Langfuse()
    trace = lf.get_trace(trace_id)
    observations = trace.observations

    stats = parse_trace(trace_id)
    risks = detect_risks(stats, observations)

    return {
        **stats,
        "risks": risks,
        "has_critical_risk": any(r["severity"] == "ERROR" for r in risks),
    }

```

使用示例

```

result = parse_trace_with_risks("trace_id_here")

if result["has_critical_risk"]:
    print("🚨 发现高危风险，需要人工复查")
    for r in result["risks"]:
        if r["severity"] == "ERROR":
            print(f"🔴 [{r['rule']}] {r['detail']}")

# 典型输出（发现工具循环）：
# 🚨 发现高危风险，需要人工复查
# [tool_loop] 工具 'query_logistics' 连续被调用 ≥2 次, Agent 可能陷入死循环

```

`parse_trace` 负责数据提取，`detect_risks` 负责规则判断，两者解耦。后续新增规则，只扩展 `detect_risks` 即可，不影响解析逻辑。

本节核心观点

三层分析对应三类指标： – 表面层 → `execution_chain`、`empty_tool_input`：调用序列和参数是否正确 – 效率层 → `tool_call_count`、`tool_loop`、`total_tokens`：有没有绕路和循环 – 风险层 → `llm_slow_call`、`too_many_steps`、`has_critical_risk`：有没有危险行为

两个函数，职责分明： – `parse_trace`：提取 7 项统计数据，是后续所有分析的原材料 – `parse_trace_with_risks`：解析 + 4 条规则检测一步完成，`has_critical_risk=True` 即触发人工复查

评分器告诉你结果，Trace 告诉你过程。两者都不看，你是瞎子；只看结果，你是独眼龙。

实操

用 Ch21 接入 Langfuse 后采集到的 Trace，完成以下验证：

第一步：用 `parse_trace` 跑一遍，检查 `execution_chain` 的顺序是否符合预期，`total_tokens` 是否在合理范围。

第二步：用 `parse_trace_with_risks` 跑一遍，看有没有规则触发。如果有 WARNING，结合 Trace 判断是真实问题还是阈值需要调整。

第三步：如果发现 `has_critical_risk=True`，打开 Langfuse 平台上对应的 Trace，定位是哪一步触发了规则，写一句话：“风险发生在 Step __ 的 __ 环节，原因是 __。”

下节预告

Trace 读完了，基线建好了，用例设计清楚了，评测集也有了。前面 21 章把工程方法全铺开了。如果今天是 Day 1，第一步做什么？Anthropic 团队走过的 8 步，是你最短的上手路径。

Anthropic 8步路线图——今天就能动手

学习目标：掌握 Anthropic 亲历的评估启动 8 步路线图，理解每步的核心价值和常见陷阱，学会从 Step 0 直接开始——今天就能落地，不需要等”准备好”。

引言

别等了。你不需要 1000 个任务才能开始。

10 个真实场景的任务 + 一套评分规则 = 你的第一版评估套件。Anthropic 也是这么开始的。

工程师等待”准备好再开始评估”的理由永远层出不穷：数据不够多、场景没覆盖全、评分逻辑还没想清楚……结果是 Agent 已经上线了，评估套件还在”计划中”。

前面几篇的工程方法全铺开了。今天是 Day 1——第一步做什么？

本章是工程篇的总结，也是进阶篇的起点——先给你一张完整路线图，下一章开始讨论让评估更可信的深度技巧。

Anthropic 8 步路线图

这不是我设计的框架，是 Anthropic 团队亲历的路径。他们构建 Claude Code、构建内部评估体系，走的就是这 8 步。

Step 0：尽早开始——20 个失败案例就够了

不要等”有足够数据”。Anthropic 用 20 个内部发现的真实失败案例起步——不是 200 个，不是 2000 个，是 20 个。

为什么 20 个就够？因为你的目标不是”覆盖所有场景”，而是”让系统可以运转”。10–20 条高频失败案例，已经能告诉你 Agent 最突出的问题在哪里。

Step 1：把每次手工测试固化为可复现任务

你现在怎么验证 Agent 好不好？打开 Agent，输入一段测试，看看输出。这是对的——但这件事做完就扔了。

每次手工测试之后，把它存下来：输入是什么、预期行为是什么、这次的实际结果。这就是可复现任务。下次你改了 Prompt，还是对着同一批任务跑——不是靠记忆觉得“好像好多了”，而是靠数字说话。

手工测试是黄金。但手工测试如果不沉淀，是会蒸发的。

Step 2：附带参考解法，写无歧义任务

任务描述必须能让两个不同的评估者得出一致结论。

一条模糊任务：“帮我查一下物流，回复要好一点。”一条清晰任务：“用户输入‘帮我查订单 12345 的物流’，Agent 应调用物流 API 并返回：物流单号、当前状态、预计到达时间。回复语气友好，不应包含推销内容。”

模糊任务的问题不是“写起来麻烦”，是“两个人标注会给不同答案”——评估结果因人而异，等于没有评估。附带参考解法（标注为什么这个输出是“通过”或“不通过”），是对齐判断标准最直接的方式。

Step 3：构建平衡用例集——正例 + 负例都要有

大多数人的本能是堆“Agent 应该做什么”的正例。但 Ch19 讲过：你需要同等比例的“Agent 不应该做什么”的负例。

不能触发物流查询的输入（闲聊、骂人、不相关请求）——这些负例，和标准查询的正例同样重要。只有正例，Agent 学会的是“怎么做对”；有了负例，Agent 才能学会“怎么不做错”。

Step 4：构建稳健框架——环境隔离不是可选项

这一步 Anthropic 吃过亏。

他们的某个 Agent 评估成绩好得离谱——调查后发现它在偷看之前试验留下的 git 历史。Agent 并没有真的解决问题，它只是“记住了上次的答案”。

环境隔离的三个核心检查： – **git 历史**：每次试验前清理，确保 Agent 看不到之前的执行记录 – **资源状态**：数据库连接、文件句柄、网络连接是否全部重置 – **临时文件**：上次试验留下的中间文件是否清理干净

每次试验必须是“第一天上班的新人”——没有记忆，没有历史，只有任务。

Step 5: 精心设计评分器——优先用确定性规则

你的第一反应可能是直接上 LLM 裁判。但 Anthropic 的实践建议相反：**能写代码规则的，先写代码规则。**

代码规则是确定性的——同样的输入永远给出同样的判断。LLM 裁判是概率性的——它可能今天说“通过”，明天说“不通过”。

评分器的优先级：1. 精确匹配（字段存在？参数正确？）→ 代码 2. 格式检查（JSON 合法？必填字段全了？）→ 代码 3. 语义质量（回复是否得体？是否有多余行为？）→ LLM 裁判 4. 无法量化的边界判断 → 人工

Step 6: 坚持读 Trace——这是最容易被跳过的步骤

数字在飙升，通过率从 70% 到 85% 到 90%——你觉得很好，于是停止读 Trace 了。

这是最危险的时刻。

Trace 是你发现评分器盲区的唯一方式（Ch21 讲过的那条客服 Agent 案例——退款了，评分器给了 4/5，Trace 里有两个高风险行为）。评分器只会告诉你它能看到的。Trace 告诉你它看不到的。

Anthropic 团队的实践：无论通过率多高，每月固定抽 20–30 条 Trace 手动阅读。不是因为数字不好看，是因为数字好看不等于 Agent 行为健康。

Step 7: 监控饱和——通过率稳定后毕业

这套评估任务一直在用，最终有一天——你发现通过率稳定在 100%，改什么都是 100%，这套任务已经区分不出“好”和“更好”了。

这是毕业信号，不是停工信号。

毕业 = 这批任务从“能力评估”升级为“回归评估”，进入 CI 每次都跑（守住底线），同时创建新的更难的能力评估任务（探索新上限）。

测试套件只增不减——每解决一个问题，就增加一道防线。

Step 8: 持续维护和开放贡献

评估体系不是一次性工程，它和 Agent 一起生长。

维护三件事：– 定期用新的线上失败案例扩充任务集 – 每次模型升级后用人工确认重新校准基线 – 把“修好的问题”对应的用例固化进回归套件

如果你的团队有多个人在使用这个 Agent，鼓励每个人把自己遇到的失败案例贡献进任务集。众人拾柴——每一个新失败案例，都在让评估体系更强一点。

 8 步路线图速查表

Step 0	尽早开始	→ 20 个失败案例，今天就能起步
Step 1	固化手工测试	→ 每次手工测试之后，把它存成可复现任务
Step 2	写无歧义任务	→ 附带参考解法，两个评估者对同一任务结论一致
Step 3	平衡用例集	→ 正例 + 负例（不该触发的场景同样重要）
Step 4	环境隔离	→ git 历史/资源状态/临时文件，每次试验重置
Step 5	优先确定性评分器	→ 能写代码规则的不用 LLM 裁判
Step 6	坚持读 Trace	→ 通过率飙升时更要读，而不是少读
Step 7	监控饱和毕业	→ 通过率稳定→回归评估+创建新能力评估任务
Step 8	持续维护	→ 线上失败案例回流，模型升级重新校准基线

 每步最常见的错误

步骤	最常见的错误
Step 0	“再等一等，数据不够多”
Step 1	测完就忘，不沉淀成文件
Step 2	任务描述模糊，不同人标注结果不同
Step 3	99% 是正例，1% 是负例
Step 4	跳过环境隔离，结果 Agent 在”作弊”
Step 5	一上来就用 LLM 裁判，代码规则能解决的不用代码解决
Step 6	通过率好了就停止读 Trace
Step 7	永远在用同一批任务，没有毕业机制
Step 8	线上出了问题，没有回流到任务集

⚠️ 这 8 步的顺序很重要

你不能跳过 Step 4 直接做 Step 5——环境不干净，评分结果不可信。

你不能跳过 Step 2 直接做 Step 3——任务描述模糊，正负例的标注就会有歧义。

但你可以在 Step 0 就同时做 Step 1 和 Step 2——边收集失败案例，边把它们固化成清晰的任务描述。这两步是自然并行的。

按顺序走，但别以为要每步都“完美”才能进下一步。 Anthropic 的 Step 0 用的是 20 条很粗糙的案例，不是 20 条精心设计的完美用例。够用就走下去。

🎯 本节核心观点

Anthropic 8 步路线图： – Step 0 尽早开始（20 个任务） – Step 1 固化手工测试 – Step 2 写无歧义任务（附带参考解法） – Step 3 平衡用例集（正例 + 负例） – Step 4 环境隔离（防 Agent 作弊） – Step 5 优先确定性评分器 – Step 6 坚持读 Trace（通过率高也要读） – Step 7 监控饱和毕业（能力评估→回归评估） – Step 8 持续维护（线上回流+模型升级重校准）

核心原则：别等准备好。 Anthropic 也是从 20 条失败案例起步的。

这 21 章讲的所有方法，最终目的只有一个：让你对 Agent 有可靠的判断力——不是靠直觉，不是靠运气，而是靠数据和过程。

实操

今天下班前，完成你的 Step 0-2：

第一步（Step 0）： 打开你的 Agent，想一想最近让你印象最深的 3 次失败——Agent 做出了你没预料到的错误行为。把这 3 条写下来。这就是你的第一批任务集。

第二步（Step 1）： 对每条失败案例，写出它的可复现形式：输入是什么、Agent 的错误行为是什么、正确行为应该是什么。

第三步 (Step 2)：检查你写的任务描述——如果让同事来看，他能不能根据描述独立判断”通过”还是”不通过”？如果不能，补充参考解法，直到答案无歧义。

这 3 条任务，就是你的第一版评估套件。下周再补 3 条，下下周再补 3 条——评估体系就这么长起来了。

下节预告

工程篇结束了。进阶篇开始——第一个 hard 问题：LLM 裁判为什么在长文档里犯糊涂？你把 50 条记录、20 条规则、10 个范例全扔进 Prompt，LLM 把所有人的 ID 都搞混了。窗口大 ≠ 脑子好用。

上下文工程——LLM裁判为什么在长文档里犯糊涂

学习目标：理解 LLM 裁判在长上下文中的四种失效场景，掌握五种应对策略，学会用“瘦身”思维优化评估 Prompt——让评分准确率从 72% 提升到 89%。

引言

LLM 号称支持百万 Token 窗口。你把 50 条记录、20 条规则、10 个范例全扔进 Prompt——LLM 把所有人的 ID 都搞混了。

窗口大 ≠ 脑子好用。

工程师在构建 LLM 裁判时，最常犯的错是“信息越多越准确”。你觉得给 LLM 更多上下文，它判断会更可靠。结果是——上下文一超过某个量，LLM 开始张冠李戴，把 A 的问题归到 B 头上，把首段的规则忘在脑后，把中段的例子和末段的数据搅在一起。

这不是模型的问题，是你的 Prompt 设计问题。

上一章教你给评估搭了路线图，这一章解决路线图上的第一个减速带：你把所有信息一股脑塞给 LLM 裁判，它反而变傻了。瘦身不是为了省钱——是为了让裁判别犯糊涂。

💡 四种上下文失效场景

失效 1：上下文混淆（张冠李戴）

最常见，最难察觉。

你的评估 Prompt 里有 50 条客服对话记录，每条有独立的 session_id。LLM 在处理第 40 条时，偶尔会把第 38 条的错误归到第 40 条身上——因为两个 ID 长得相似（session_abc123 和 session_abc124），在 50 条的长上下文中“混”在一起了。

评分结果看起来”差不多”，但仔细核查你会发现：大量的错误归因不是 LLM 判断力不行，是它把账记错了。

失效 2：上下文丢失（开头信息被边缘化）

研究证明，LLM 对 Prompt 开头和结尾的注意力远高于中段。你在 Prompt 第一段写了最重要的评分规则，LLM 跑到第 30 条对话时，那条规则已经”淡出视野”了。

结果：越靠后的对话，越容易忽略前面的关键约束。

失效 3：上下文冲突（近因 vs 首因）

你的 Few-Shot 范例在 Prompt 中段，评分规则在 Prompt 开头。当范例和规则略有出入时，LLM 倾向于跟随”更近的那个”——Few-Shot 范例覆盖了开头的规则。

你以为 Few-Shot 在帮你，实际上 Few-Shot 在悄悄改写你的规则。

失效 4：上下文干扰（无关信息污染判断）

每条对话记录里带了 20 个 metadata 字段：创建时间、设备类型、用户 ID、标签.....这些字段和”回复是否得体”完全无关，但它们占了大量 Token，把真正有用的信息稀释了。

LLM 的注意力是有限资源。无关信息越多，有用信息越难被关注到。

真实案例：从 72% 到 89%

一个评估 Prompt 的实际情况：

评估对象：50 条客服对话记录
每条记录包含：session_id、时间戳、用户ID、设备类型、
完整对话内容、物流单号、API调用记录.....
评分规则：20 条
Few-Shot 范例：10 个

总 Token 量：约 15,000 Token
评分准确率：72%（与人工评分对比）

具体失效表现：LLM 把 session_abc123 的物流查询超时错误归到了 session_abc124 上——两个 ID 相邻，内容相似，在长上下文中发生了混淆。

瘦身方案三步走：

第一步：删冗余字段

每条对话记录从 200 Token 压缩到 80 Token——删掉时间戳、设备类型、用户 ID 等与评分无关的字段。只保留：session_id、对话内容、相关 API 调用结果。

第二步：长 ID 映射短编号

```
# 处理前
session_id: "session_abc123"
session_id: "session_abc124"

# 处理后
S1 → session_abc123
S2 → session_abc124
```

这一步消灭了“张冠李戴”的根源。

第三步：拆分复合指令

原来一次 LLM 调用评两件事：“功能是否正确” + “回复是否得体”。拆成两次独立调用，每次只评一个维度。

结果：评分准确率从 72% → 89%。

五种应对策略

策略 1：逻辑拆分

把“评 N 条记录”拆成 N 次独立调用，每次只处理 1 条。代价是调用次数增加，优势是完全消除上下文混淆。

如果不能拆到 1 条，至少把复合任务拆开——功能评估一次调用，体验评估一次调用。

策略 2：内容精简（控制在 5 万 Token 内）

实践建议：单次 LLM 裁判调用控制在 5 万 Token 以内。每条记录只保留评分必要的字段，其余全删。

能用 ID 替代长字符串的，全部替代。

策略 3：强制约束输出格式

要求 LLM 裁判输出纯净 JSON，而不是自然语言。JSON 格式强制了 LLM 结构化思考，减少模糊归因：

```
{
  "session_id": "S1",
  "score": 3,
  "reason": "未按规则重试超时请求"
}
```

策略 4: Few-Shot 放对位置

Few-Shot 范例要么放在 Prompt 最开头（利用首因效应），要么放在被评估内容的紧前方（利用近因效应）。

放在 Prompt 中段是最坏的选择——既不够近，也不够先。

策略 5: 利用首因效应

最重要的评分规则放在 Prompt 第一段，不是最后一段。LLM 对第一段的记忆远比中段稳定。如果你有 3 条核心规则和 17 条辅助规则，把 3 条核心规则提到最前面，单独成段，加粗标注。

! 核心认知

降低 Token 量 ≠ 降低信息量。

删掉的是冗余字段、无关 metadata、过长的 ID 字符串。保留的是真正参与评分的内容。

绝大多数评估 Prompt 的问题不是信息不够，是信息太多——而且质量参差不齐，真正有用的淹没在噪音里。

瘦身不是损失，是聚焦。聚焦才能让 LLM 把有限的注意力放在真正重要的地方。

🎯 本节核心观点

四种失效场景： – 上下文混淆：张冠李戴（相似 ID 在长上下文中混淆） – 上下文丢失：开头规则被遗忘 – 上下文冲突：Few-Shot 覆盖了评分规则 – 上下文干扰：无关字段稀释注意力

五种策略： 1. 逻辑拆分（1 条 = 1 次调用） 2. 内容精简（控制 5 万 Token 内，删冗余字段） 3. 强制约束（纯净 JSON 输出） 4. Few-Shot 位置优化（首位 or 紧前方） 5. 首因效应（核心规则放第一段）

黄金原则： 窗口大 ≠ 脑子好用。评估 Prompt 的质量，不取决于塞了多少信息，而取决于去掉了多少干扰。

实操

对你的 LLM 裁判 Prompt 做一次”瘦身体检”：

第一步： 数一下当前评估 Prompt 的 Token 量。超过 5 万了吗？

第二步： 检查每条记录里的字段——哪些字段和评分标准无关？删掉它们。计算瘦身后的每条记录的 Token 量（目标：压缩 30% 以上）。

第三步： 看你的 session_id 或其他 ID——是否超过 10 个字符？如果是，写一个映射表，把它们替换成 S1、S2、S3 这样的短编号。

第四步： 回跑同一批评估任务，对比瘦身前后的评分一致性。

下节预告

Prompt 设计好了，但有一个更底层的问题：LLM 裁判要不要开深度思考模式？让它”多想十分钟”，有时候值回票价，有时候纯属浪费——判断标准是什么？

深度思考模式——你该不该让它多想一想

学习目标：理解深度思考模式与普通 CoT 的本质区别，掌握“该不该开”的判断框架，学会用准确度/耗时/成本三角衡量深度思考的投入产出比——不是每个评估任务都值得“多想一想”。

引言

深度思考模式就像让裁判在打分前先自言自语十分钟。有时这十分钟值回票价，有时纯属浪费。

怎么判断？

很多工程师的默认选择是“开就开吧，更准一点总没错”。错了。深度思考不只是“更慢一点”，它是慢数倍、贵数倍——如果你有 500 条对话要评估，深度思考可以把成本和时间都乘以 10。

所以问题不是“深度思考好不好”，而是“这个任务值不值得”。

上一章学会给 LLM 裁判瘦身——删冗余、压缩 Token。这一章解决相反的难题：有一类任务恰恰需要“多花 Token 想一想”，怎么区分这两种场景？

💡 深度思考 ≠ 普通 CoT

先搞清楚这两件事的区别，才能知道什么时候该用哪个。

普通 CoT (Chain of Thought)

你在 Prompt 里写：“请一步一步思考，然后给出答案。”LLM 按照你的指示，把推理过程写出来。

这是你控制的外显推理——LLM 输出了推理链，你能看到它怎么想的。

深度思考模式 (Extended Thinking)

不是你在 Prompt 里加一句话。这是一个模型层面的开关——模型在生成最终回答之前，有一个独立的”思考阶段”，进行多轮内部推理，对自己的推理过程做自我批判和验证。

深度思考的五步内部流程：1. **问题解析**：把问题拆解成子问题 2. **多路径推理**：同时考虑多种可能的判断方向 3. **逐步验证**：对每一条推理路径做正确性检查 4. **自我批判**：主动寻找自己推理中的漏洞 5. **最终整合**：综合多路径结论，输出最终判断

这比普通 CoT 深得多。也贵得多——思考过程本身也消耗 Token，而且不会出现在你的最终输出里，你只看到结论，却已经付了思考的费用。

两个任务的对比实验

同一个客服 Agent 评估场景，两类任务：

任务 A：退款操作检测

任务描述：这条对话里，Agent 有没有执行退款操作？
评估维度：是/否的二元判断

代码评分器：检查 Trace 里有没有 refund_api 的调用记录。精确，0 误差。

如果用代码能解决，开深度思考就是浪费。这种”判断是什么”的任务，LLM 裁判本身都是多余的——更别说深度思考了。

任务 B：回复得体性评估

任务描述：用户情绪激动、语气强烈，Agent 在这个时机的回复——是否得体？是否在正确的时间说了正确的话？
评估维度：多维度语义理解 + 语境判断 + 情感敏感性

关闭深度思考：评分与人工评分的 Spearman 相关性 = 0.65

开启深度思考：Spearman 相关性 = 0.82

提升了 0.17。这个任务值得多花 10 倍的时间和成本。

🤖 判断框架：开还是不开？

核心问题只有一个：

这个任务需要”理解为什么”，还是只需要”判断是什么”？

任务类型	深度思考	理由
字段存在性检查	✗ 关闭	代码规则能解决，不需要 LLM
格式合法性验证	✗ 关闭	代码规则能解决，不需要 LLM
事实查询匹配	✗ 关闭	精确匹配，无需推理
情感得体性判断	✓ 开启	需要多维度语境理解
边界道德判断	✓ 开启	需要权衡多个视角
复杂意图识别	✓ 开启	需要理解潜台词和隐含意图
风险行为识别	✓ 开启	需要推理行为后果

一个简单的判断规则：

如果你能写出精确的规则或代码来判断 → 关闭深度思考（用代码）

如果人工标注者也需要”想一想”才能判断 → 开启深度思考

⚠️ 深度思考的代价

时间代价

响应时间通常慢 3–10 倍。500 条评估任务，原来 10 分钟跑完，开深度思考可能需要 1–2 小时。

成本代价

思考过程消耗的 Token 也计费（即使你看不到这些 Token）。实际成本通常是 3–10 倍。

TPM 配额代价

思考 Token 也占用 TPM（每分钟 Token 限额）。高并发评估时，深度思考容易把你的配额打满，导致限速。

这三个代价叠加的结果：如果你的评估任务 80% 是”判断是什么”类型，全开深度思考，你在把 80% 的成本花在不需要深度思考的地方。

正确策略：混合模式

代码评分器 ← 精确匹配任务（字段/格式/事实）
LLM 标准模式 ← 需要语义理解但不复杂的任务
LLM 深度思考 ← 需要多维语境判断的任务（占 20%-30%）

本节核心观点

深度思考 ≠ 普通 CoT： – CoT 是你控制的外显推理 – 深度思考是模型内部的五步自我批判循环（解析→多路径→验证→批判→整合）

开启条件： – 需要”理解为什么”→ 开 – 只需要”判断是什么”→ 关

代价三角： 时间 × 成本 × TPM 配额——全部慢/贵/占用多

实践原则： 分层使用。代码解决的不用 LLM，LLM 能解决的不开深度思考，只有真正需要多维推理的任务才值得”多想一想”。

一句话判断： 如果人工标注者也需要”想一想”，就值得开。如果答案一眼就能看出来，就是浪费。

实操

对同一条评估任务，做一次 A/B 对比：

第一步： 选一条评估任务（优先选”回复得体性”或”意图识别准确性”类型的，而非字段检查类）。

第二步： 关闭深度思考，跑 5 次，记录： – 平均评分 – 与你的人工判断一致的次数 – 平均耗时

第三步： 开启深度思考，同一条任务跑 5 次，记录相同指标。

第四步：计算这个任务值不值得开：准确度提升多少？成本增加了多少倍？你的任务量乘以成本增量，可接受吗？

下节预告

深度思考解决了”判断质量”的问题。但还有一个问题没解决：同一个任务跑 10 次，成功 7 次——这 70% 算过关吗？非确定性场景下，你需要统计证据，而不是一次结论。

pass@k与pass^k——非确定性场景下的统计证据

学习目标：理解 pass@k 和 pass^k 的本质区别，掌握如何用这两个指标区分“峰值能力”和“工程稳定性”，学会根据不同容错要求设置稳定性阈值——让统计证据代替单次直觉。

引言

同一道题做 10 次，做对 3 次——不是运气好叫能力强，是概率在帮你。真正的过关是 10 次都做对。

很多工程师只跑一次评估。跑通了，觉得好。跑一次没过，觉得有问题。这是直觉驱动，不是证据驱动。

LLM 是非确定性系统。同一个输入，每次输出略有不同。“跑一次通过”和“每次都通过”是两件完全不同的事。如果你只跑一次就下结论，你在用运气代替数据。

💡 pass@k 和 pass^k 是什么

pass@k：k 次里至少通过一次

pass@10 = 10 次执行中，至少有 1 次通过的概率

衡量的是：这个 Agent 有没有做好这件事的能力？

$\text{pass@10} = 90\%$ ，意味着 10 次里基本都能通过——能力没问题。

$\text{pass@10} = 50\%$ ，意味着 2 次里大约 1 次通过——Agent 知道正确答案，但不稳定。

pass^k：k 次全部通过

pass^{10} = 10 次执行中，每一次都通过的概率

衡量的是：这个 Agent 能不能稳定地做好这件事？

$\text{pass}^{10} = 90\%$ ，意味着有 90% 的把握本次 10 次运行全部通过——生产级稳定性。

$\text{pass}^{10} = 10\%$ ，意味着 10 次里只有约 1 次全过——不稳定，不能上生产。

一个案例：能力够，稳定性不足

客服 Agent 执行物流查询任务 10 次：

```
第 1 次: ✓ 通过
第 2 次: ✓ 通过
第 3 次: ✓ 不通过 (API 超时后未重试, 直接退出)
第 4 次: ✓ 通过
第 5 次: ✓ 通过
第 6 次: ✗ 不通过 (参数传入了错误的 order_id 格式)
第 7 次: ✓ 通过
第 8 次: ✓ 通过
第 9 次: ✓ 通过
第 10 次: ✓ 通过
```

计算结果：

```
pass@10 ≈ 100% (10 次里 8 次通过, 至少 1 次成功——确定有能力)
pass^10 = 0% (本次 10 次未全部通过。若用单次通过率估算:  $0.8^{10} \approx 10.7\%$ )
```

诊断：

- 如果只看 $\text{pass}@10 \approx 100\%$ ，你会觉得 Agent 很棒。
- 如果只看 $\text{pass}^{10} = 0\%$ （或估算值 10.7%），你会觉得 Agent 不可靠。
- 两个指标一起看：Agent 有能力做好，但工程稳定性不足——需要优化的是重试机制和超时处理，而非 Prompt 或模型。

进一步定位：

第 3 次失败（API 超时未重试）和第 6 次失败（参数格式错误）——都是工程层面的问题，不是 Prompt 层面的问题。

需要优化的是：重试机制和参数校验逻辑，而不是 Prompt 或模型。

两个指标一起看，才能区分”能力问题”和”稳定性问题”——这个区分直接决定了你该优化什么。

🤖 实践中怎么用

能力评估阶段（探索上限）：关注 $\text{pass}@k$

你在测试一个新功能，想知道模型能不能做到。

用 $\text{pass}@k$ ——只要 k 次里有一次做到了，说明这件事可行。你接下来的任务是让它更稳定，而不是判断可不可行。

回归评估阶段（守住下限）：关注 pass^k

你在保护一个已上线的功能，确保它不退化。

用 pass^k ——每次都要通过，才算守住了底线。一次失败 = 退化信号，需要调查。

稳定性容忍阈值：不同场景，标准不同

以下阈值来自工程实践经验，建议以”一次失败的最坏后果”为标准倒推调整：

场景类型	稳定性要求	推荐指标
关键决策（退款/权限变更）	$\text{pass}^k \geq 99\%$	近零容忍
核心业务流程（订单查询）	$\text{pass}^k \geq 90\%$	低容忍
辅助分析（总结/建议）	$\text{pass}^k \geq 60\%$	中等容忍
创意生成（文案/配图建议）	$\text{pass}^k \geq 40\%$	高容忍

关键决策类 Agent，不要用平均通过率来衡量——用 pass^k ，用近零容忍。

⚠️ 跑多少次才够？

最少 5 次，建议 10 次。

5 次是最低置信度要求：如果 5 次全过， pass^5 有统计意义；如果 5 次里有 3 次失败， $\text{pass}@5$ 能告诉你峰值能力。

如果是关键决策类 Agent，跑 20 次——稳定性要求越高，需要的样本越多。

单次评估的结论是脆弱的。一次通过可能是运气，一次失败可能是偶发。统计证据才是可以信任的。

本节核心观点

两个指标： – pass@k: k 次至少 1 次通过 → 峰值能力（探索上限） – pass^k: k 次全部通过 → 工程稳定性（守住下限）

使用场景： – 能力评估阶段 → pass@k – 回归评估阶段 → pass^k

容忍阈值四级： – 关键决策类 → 近零容忍 (pass^k ≥ 99%) – 核心业务 → 低容忍 (≥ 90%) – 辅助分析 → 中等容忍 (≥ 60%) – 创意类 → 高容忍 (≥ 40%)

只跑一次就下结论，是在用运气代替数据。LLM 是非确定性系统——你的评估结论也必须是统计性的。

实操

对同一个评估任务执行 5 次（或 10 次），计算两个指标：

第一步：选一个你认为“基本没问题”的任务，跑 5 次，记录每次是通过还是失败。

第二步：计算 pass@5（5 次里至少 1 次通过）和 pass^5（5 次全部通过）。

第三步：对照稳定性容忍阈值表——你的 Agent 属于哪类场景？当前的 pass^5 达标了吗？

计算提示： pass@k 的判断标准是 k 次中至少 1 次通过（能做好吗？）；pass^k = k 次全部通过（每把都稳吗？）。用 Python: `pass_at_k = sum(results) >= 1` , `pass_pow_k = sum(results) == k` 。

第四步：如果 pass@5 高但 pass^5 低，定位失败的那几次——是 Prompt 问题、工程问题还是模型本身的随机性？

下节预告

你有 50 条 Bad Case。一条一条修？修到第 5 个你会发现前 3 个的修复互相冲突。下一章教你 Bad Case 系统化反推三步法——把 50 条摊在桌上，找出那个修一个消灭 20 个的系统性问题。然后走进阿里云，看方法论在真实场景下怎么落地。

实战案例——阿里云AI评测AI对话型Agent

学习目标：（1）掌握 Bad Case 系统化反推三步法——从散乱的失败案例中找出系统性根因；（2）通过阿里云智能客服团队的真实案例，理解对话型 Agent 评估的完整工程架构——“评估-诊断-优化”三段闭环，以及让 AI 当质检员的五大工程技巧。

引言

前 26 章我们建立了完整的方法论体系。但在进入真实案例之前，还有一件事要做——你手里攒了一堆 Bad Case，怎么从里面挖出真正值得修的问题？

本章前半部分给你 Bad Case 反推三步法。后半部分带你走进阿里云，看方法论在真实场景下怎么变形、怎么填坑。

Ch17 的闭环策略说过“E2E 评测 → 失败归因 → 专项优化”。但“失败归因”这一步具体怎么做？答案在这里。

前半部分：Bad Case 系统化反推 —— 从“这个 case 没过”到“这个能力短板”

为什么逐个修是死循环？

你有 50 条 Bad Case。逐条看：

- case 1: “用户骂人时 Agent 还在推销新品”
- case 2: “用户说‘我要投诉’被当成‘我要查询’”
- case 3: “物流 API 超时后直接退款”
- case 4: “多轮对话中忘了用户前面说的订单号”
-

看起来毫不相关。一条一条修？修到第 5 个你会发现前 3 个的修复互相冲突——让 Agent 对情绪敏感了，它却把正常的催促也当成投诉，通过率反而下降。

单个 Bad Case 是症状，失败模式才是病因。

反推三步法

第一步：归类抽象——按失败模式分类，不是按表面现象

把 50 条 Bad Case 摊在桌上，不看”这条说了什么”，看”这条为什么失败”：

失败模式	典型案例	占比
意图识别错误	情绪型输入（骂人/说反话/表达愤怒）被当成常规查询	45%
工具调用失败	API 超时/参数错误，缺乏重试机制	30%
上下文管理	多轮对话中丢失前面提到的订单号、偏好	15%
回复质量	语气不当/多余推销/信息不完整	10%

归类的关键：**合并同类项**。你看到的不再是 50 个孤立的失败，而是 4 种失败模式。

第二步：帕累托定位——20% 的失败模式贡献 80% 的 Bad Case

画出帕累托图：意图识别错误（45%）+ 工具调用失败（30%）= 75% 的问题。

这意味着：**修好前两种失败模式，就能消灭 75% 的 Bad Case**。不需要修 50 个，只需要解决 2 个系统性问题。

第三步：反推策略优化——修一个失败模式，消灭一片 case

不是”在 Prompt 里加一条规则处理骂人场景”（这是修单个 case），而是问：**为什么意图识别模块整体上对负面情绪不敏感？**

定位到系统性问题：意图识别模块的处理顺序是”先判断意图→再生成回复”，没有”先检测情绪→再判断意图”的步骤。用户的愤怒情绪被正常路由到了查询流程。

反推优化：在 Prompt 中增加”先检测情绪→再判断意图”的推理顺序。优化后意图识别类 Bad Case 减少 65%。

关键洞察：修一个推理顺序（一行 Prompt 改动），消灭了 $45\% \times 65\% \approx 30\%$ 的全部 Bad Case。

与 Ch17 闭环的关系

Ch17 的闭环策略是宏观流程：E2E 评测 → 失败归因 → 专项优化 → 二次验证。

Bad Case 反推三步法就是其中“失败归因”这一步的详细操作手册。两者配合使用：– 闭环策略告诉你**什么时候做**归因分析 – Bad Case 反推告诉你**怎么做**归因分析

实操（Bad Case 反推部分）

现在就做：

1. 收集你 Agent 的 10 条失败案例（如果没有，回看 Ch1 盲测和后续各章的实操记录）
 2. 按失败模式归类——不看表面现象，看失败原因
 3. 画帕累托图：哪类失败模式占比最高？
 4. 对 Top 1 失败模式，定位系统性问题并设计优化方案（不是修单个 case，是修模式）
 5. 和 Ch17 的闭环计划对照——这次反推的优化点能不能接入那个闭环？
-

后半部分：阿里云”AI 评测 AI”——对话型 Agent 的真实落地

为什么对话型 Agent 的评估最难？

对话型 Agent 的评估有一个根本性难题：**没有标准答案**。

用户问”我的快递怎么还没到？”——”快递预计明天到””帮您加急处理了””正在为您查询”都是合法回复。哪个最好，取决于情境、情绪、业务规则。

这和工具型 Agent（步骤对不对？） 、内容生成型 Agent（图好不好看？） 都不同——对话的好坏本身就是一个需要语义理解的判断。

这意味着：代码评分器（Ch7）只能处理格式和边界，语义质量必须靠 LLM 裁判（Ch8）——而 LLM 裁判的可靠性又是最难保证的。

阿里云面对的就是这个问题。

架构：评估-诊断-优化三段闭环

阿里云的系统用三段架构解决这个问题：

```
graph TD
    A[线上客服对话] --> B["【评估】— 用大 LLM 对小 LLM 的回答打分"]
    B --> C["【诊断】— 找出 BadCase 的根因（属于哪类失败模式）"]
    C --> D["【优化】— 自动生成改进数据，补充知识库"]
    D --> E["【二次验证】— 优化后回归，确认问题已修复"]
```

表面上看，这和 Ch17 的闭环策略是一样的——发现→归因→优化→验证。

但阿里云做了一件额外的事：**优化这一步本身也是自动化的**。诊断到“知识库缺乏 X 场景信息”时，系统会自动在互联网上检索相关数据，补充到知识库里，再触发重新评估。

这不是纯粹的评估系统，而是一个**自愈的 Agent 运营系统**。

五大工程技巧

技巧一：正负向混合评判（解决 LLM 裁判的单边偏差）

Ch11 讲过 LLM 裁判的位置偏见——先看到正面内容，后面的负面问题容易被忽视。

阿里云的解法：**强制 LLM 先找优点，再找缺点，最后综合权衡**。

```
评判 Prompt 结构：
1. 先列出这条回复做得好的地方（优点 2-3 条）
2. 再列出这条回复的问题点（缺点 1-3 条）
3. 综合上述分析，给出 1-5 分的整体评分
```

这个顺序不是随意设计的——它强迫 LLM 在脑子里同时持有正反两面的信息，再做综合判断，而不是看到前几句好就直接打高分。

实测效果：Bad Case 的召回率从 68% 提升到了 85%+。

技巧二：两级业务类型/场景映射（解决泛化失效）

通用的评分标准在特定业务场景下会失效。

比如：快递时效类问题的回答要求精准（“预计 3 天内到”比“快了”好），但商品咨询类问题可以更宽泛（“这款适合你”比“规格参数是 XXX”更有体验感）。

用同一套 Rubric 评所有场景，LLM 裁判会判错。

阿里云的解法：**两级映射**： – **一级**：业务大类（物流/退款/商品/活动） – **二级**：具体场景（物流-查询/物流-催单/物流-投诉）

每个具体场景对应一套专属 Rubric。评估前，先判断这条对话属于哪个场景，再调用对应 Rubric 打分。

这样 LLM 裁判不再用一把尺子量所有事情，而是对不同场景分别做专业判断。

技巧三：多 LLM 对抗（解决单一模型偏差）

Ch11 讲的多模型对抗在这里真实落地了。

阿里云用了 2-3 个不同模型家族的 LLM 同时评判同一条对话，取多数投票结果。

关键点：**必须选不同家族的模型**。用两个 Claude 对抗，因为训练数据相似，偏差方向也会相似，对抗价值不大。用 Claude + GPT + 千问，三个模型的偏差方向各不相同，多数投票才能真正降低单一偏差的影响。

代价：3 倍 API 成本，3 倍响应时间。只在关键质量判断环节开启，格式校验等用代码规则处理。

技巧四：上下文工程（解决 LLM 裁判在长对话里犯糊涂）

Ch23 的内容在这里用到了。

客服对话往往有 10-20 轮，加上评分规则、Few-Shot 示例，一次评估请求很容易超过 5 万 Token。这时候上下文混淆（session ID 混淆、前后轮信息错位）的风险急剧上升。

阿里云的处理： 1. **逻辑拆分**：功能正确性和体验质量分两次独立调用评估，不合并 2. **内容精简**：对话中的 metadata（时间戳、设备信息）在不影响评判的情况下剥离 3. **强制约束**：

输出格式锁死为 JSON, `{score: int, reason: str, evidence: str[]}` 4. **Few-Shot 前置**: 把示范案例放在 prompt 最前面, 利用首因效应

效果: 长对话场景下的评分一致性从 72% 提升到 89% (参考 Ch23 的案例数据)。

技巧五: Checkpoint 中断恢复 + 并发限流

Ch10 提到过这两个工程细节, 在阿里云的系统里都真实用到了。

Checkpoint 中断恢复: 大批量评估 (比如一次处理 500 条对话) 时, 如果中途 LLM API 超时或系统崩溃, 不能从头重跑——会浪费大量 API 配额。阿里云在每 50 条对话评估完成后写一次 Checkpoint, 中断后从上次 Checkpoint 续跑。

并发限流 (QPM + TPM): 评估系统自己也在调用 LLM, 如果并发太高, 会把 API 配额耗尽, 影响主系统 (AI 客服) 的正常服务。阿里云设置了独立的评估专用 API Key, 并限制了每分钟请求数 (QPM) 和 Token 数 (TPM)。

这两个细节在系统小规模验证时不会暴露, 但一旦评估规模扩大, 就会变成拦路虎。

实际效果

阿里云这套系统的量化结果:

指标	数值
Bad Case 发现准确率	85%+
根因归类准确率	80%+
人工标注量减少	~70%

这意味着——以前需要 10 个质检员每天手工审核的工作量, 现在 AI 系统每天自动完成, 人工只需处理 AI 拿不准的边界案例 (约 15-20% 的样本量)。

本节核心观点

对话型 Agent 评估的核心挑战：没有标准答案——语义质量判断必须靠 LLM 裁判，而 LLM 裁判的可靠性是最大风险点。

阿里云的三段架构：评估→诊断→优化，形成自愈的 Agent 运营系统。

五大工程技巧（以及对应的前置章节）： 1. 正负向混合评判 → 解决位置偏见（Ch11） 2. 两级业务场景映射 → 解决泛化失效（Ch12） 3. 多 LLM 对抗 → 解决单一模型偏差（Ch11） 4. 上下文工程 → 解决长对话混淆（Ch23） 5. Checkpoint + 限流 → 解决大批量工程可靠性（Ch10）

关键洞察：校准比数量重要，不是评估覆盖多少条对话，而是评估准不准。BadCase 发现准确率 85% 的意义在于：你找出的问题是真问题，而不是评估系统自己出错了。

实操

对照阿里云案例的”评估-诊断-优化”三段闭环架构：

第一步：画出你自己 Agent 的三段闭环设计图——评估环节用什么方法？诊断环节能归类几种根因类型（参考本章前半部分的 Bad Case 反推三步法——先归类失败模式，再做帕累托定位）？优化环节是手动触发还是自动触发？

第二步：阿里云的五大工程技巧中，哪一个你的系统最缺？（多数团队最先缺的是”两级场景映射”——用一套 Rubric 评所有场景的问题往往最先暴露。）

第三步：如果你要用 LLM 做质检员，写出你的评判 Prompt 的核心结构：先找优点几条？再找缺点几条？输出格式是什么？

下节预告

对话型 Agent 评估没有标准答案。但如果换成工具型 Agent——步骤对不对？参数对不对？答案其实是确定的。腾讯 TPerf 团队搭了一套扣分制 + 0 容忍的工具 Agent 评估体系，关键决策类 Agent 一次失败 = 全部不通过。下一节，看他们怎么做到的。

实战案例——腾讯TPerf智能分析工具型Agent

学习目标：通过腾讯 TPerf 性能平台智能分析 Agent 的真实案例，理解工具型 Agent 评估的核心方法——100 分扣分制、LCS 序列比对、0 容忍稳定性策略，以及面向不同研发阶段的四种执行模式。

引言

阿里云的对话型 Agent 评估，最大的难点是“没有标准答案”——什么是好的回复，需要 LLM 裁判来判断。

腾讯 TPerf 遇到的问题恰恰相反。

TPerf 是腾讯云的性能压测平台，它的智能分析 Agent 任务是：**分析性能数据，找出瓶颈，给出优化建议。**

这类任务有客观标准：分析步骤对不对？参数提取准不准？报告结论是否基于数据？

答案是确定的——这是工具型 Agent 评估最大的优势，也是最大的陷阱：**因为标准客观，你反而会把要求提得很高，然后发现 Agent 达不到。**

TPerf 团队的经验是：先用扣分制暴露最薄弱的环节，再用 0 容忍策略守住关键底线。

💡 什么是工具型 Agent 评估的核心难题？

工具型 Agent 的评估重点和对话型不同：

维度	对话型（阿里云）	工具型（TPerf）
核心问题	回答质量够不够好？（语义判断）	步骤对不对？结果准不准？（逻辑判断）
评分器	LLM 裁判为主	代码规则为主
最大挑战	没有标准答案	如何对比复杂的步骤序列
稳定性要求	允许一定波动	关键决策 0 容忍

工具型 Agent 的评估难不在“怎么打分”，而在：

1. **步骤序列比对**：Agent 的执行路径是动态的，两次跑出来的步骤顺序可能不完全一样——哪些差异是可接受的，哪些是错误？
2. **关键决策的容忍度**：如果 Agent 给出了错误的性能优化建议，工程师按这个建议操作会出事——这类 Agent 的错误代价极高。

核心方法一：100 分扣分制

TPerf 用的不是加分制（答对一条加多少分），而是从 100 分开始扣。

```
总分 100 分
├─ 步骤正确性：满分 10 分（LCS 比对，扣分）
│   ├── 工具调用序列是否与基线一致
│   └── 工具参数是否正确
├─ 效率：满分 10 分（扣分）
│   ├── Token 消耗是否超基线 1.5 倍
│   └── 工具调用次数是否超基线
└─ 报告质量：满分 80 分（扣分）
    ├── 数据准确性（分析数据是否与原始数据一致）
    ├── 分析深度（是否覆盖关键瓶颈）
    ├── 结构完整性（报告格式是否完整）
    └── 建议可执行性（优化建议是否具体可操作）
```

为什么用扣分制，不用加分制？

加分制天然有“及格就好”的倾向——Agent 只要做到基本步骤，就能积累到分数，很难暴露真正的薄弱点。

扣分制不同：**Agent 最薄弱的地方会被集中扣分，帕累托效应更明显**——哪个维度扣分最多，哪个维度就是优化优先级最高的。

🔧 核心方法二：LCS 序列比对（解决步骤评估难题）

工具型 Agent 的执行路径是一个序列：

```
基线路径：[查询 CPU 负载] → [查询内存使用] → [调用 DB 分析工具] → [生成报告]
本次执行：[查询内存使用] → [查询 CPU 负载] → [调用 DB 分析工具] → [生成报告]
```

这两个序列哪个更好？顺序不同，但内容一样。

暴力比对会说“路径不一致，扣分”——但实际上顺序不同只是实现细节，不影响最终分析结果。

TPerf 用的是 **LCS（最长公共子序列）算法**：

```
# 伪代码：LCS 比对
baseline_steps = ["查询CPU", "查询内存", "DB分析", "生成报告"]
actual_steps   = ["查询内存", "查询CPU", "DB分析", "生成报告"]

lcs_length = LCS(baseline_steps, actual_steps) # = 3 (查询CPU, DB分析, 生成报告)
step_score = (lcs_length / len(baseline_steps)) * 10 # = 7.5 分
```

LCS 比对的好处：允许步骤顺序有合理变动，只检查关键步骤是否缺失或替换。

实践中 TPerf 会设定哪些步骤是“强制步骤”（缺失即重大扣分），哪些是“可选步骤”（缺失小扣分）。

🔧 核心方法三：0 容忍稳定性策略

Ch25 讲了 $\text{pass}@k$ 和 pass^k 两个指标——能力验证阶段用 $\text{pass}@k$ ，稳定性验证阶段用 pass^k 。

TPerf 做了一个更严格的选择：关键决策类 Agent，不是取平均分，不是多数投票，而是 **0 容忍**——任一 Trial 不达标，整个用例不通过。

用公式表达：

```
通过条件 = (Trial 1 通过) AND (Trial 2 通过) AND (Trial 3 通过)
```

这等价于强制 $\text{pass}^k = 1.0$ ，即必须 k 次全部通过。

为什么这么严苛？

因为性能分析 Agent 的建议是用来指导工程师操作的。一次错误建议，工程师按指令操作，可能造成生产事故。这类 Agent 不能“平均而言是对的”，它必须“每次都对”。

这个策略不适合所有 Agent。TPerf 自己给出了适用条件： – 关键决策类 $\rightarrow 0$ 容忍 ($\text{pass}^k = 1.0$) – 辅助分析类 $\rightarrow 90\%$ 通过率即可 ($\text{pass}^k \geq 0.9$) – 探索型任务 $\rightarrow$ 至少 1 次通过 ($\text{pass}@k \geq 1$)

四种执行模式

TPerf 的评估系统设计了四种模式，对应不同研发阶段的需求：

模式	适用场景	运行范围	耗时
快速分析	单次 Prompt 调试	单模型 $\times$ 单用例 $\times$ 1 次 Trial	秒级
全量测评	发布前验证	全模型 $\times$ 全用例 $\times$ 多次 Trial	3–5 小时
按模型筛选	模型升级验证	指定模型 $\times$ 全用例 $\times$ 多次 Trial	30–60 分钟
重新评分	评分规则迭代	Trace 已缓存，只重跑评分器	分钟级

重新评分模式是其中最精妙的设计——Trace 存档后不需要重跑 Agent，只需重跑评分器。当你发现评分规则有问题（比如步骤比对太严格，顺序差异也被扣分了）时，直接修改评分规则，对历史 Trace 重新打分，就能知道这条规则的调整是否改善了结果。这让**评分器的迭代速度**和 Agent 本身的迭代解耦了——不需要每次修改规则都重跑所有 Agent。

本节核心观点

工具型 Agent 评估的优势：客观标准存在，代码规则为主，可信度高。

工具型 Agent 评估的难点：步骤序列比对、关键决策的容忍度设定。

TPerf 的三个核心方法：1. 扣分制 → 暴露最薄弱环节（帕累托效应更明显）2. LCS 序列比对 → 允许步骤顺序灵活，但关键步骤不能缺 3. 0 容忍稳定性 → 关键决策类 Agent 一次失败即全部不通过

四种执行模式的设计思路：把”跑 Agent”和”运行评分器”解耦——Trace 缓存后，评分规则可以独立迭代。

关键问题问自己：你的 Agent 是辅助类还是决策类？这决定你该用 $\text{pass}@k$ 还是 $\text{pass}^k=1.0$ 。

实操

第一步：借鉴 TPerf 的扣分制，为你的 Agent 设计一张 100 分评分表：– 列出 3–5 个评估维度 – 为每个维度分配权重（合计 100 分） – 标注每个维度的扣分规则（满足什么条件扣多少分）

第二步：你的 Agent 的执行是否有固定的步骤序列？如果有，标注哪些是”强制步骤”（缺失大扣分），哪些是”可选步骤”（缺失小扣分或不扣分）。

第三步：你的 Agent 属于什么类型？– 如果是关键决策类 → 考虑 0 容忍策略（ $\text{pass}^k = 1.0$ ）– 如果是辅助分析类 → 设定通过率阈值（如 $\text{pass}^k \geq 0.9$ ）

第四步：你的评估流程中，是否支持”重新评分”模式？Trace 是否有持久化存档？如果没有，这是你现在可以立刻改进的地方。

下节预告

对话型没有标准答案，工具型答案客观确定——那如果 Agent 的输出是”一张图”呢？AIGC 配图 Agent 的评估既不是语义判断，也不是逻辑判断，而是”好不好看”这种人人说法不一的主观判断。下一节，看 AIGC 配图团队如何从 0 搭起一套以校准为核心的评估体系。

实战案例——AIGC配图Agent评测从0到1内容生成型Agent

学习目标：通过 AIGC 配图 Agent 评测从 0 到 1 的完整过程，理解内容生成型 Agent 评估的核心挑战——主观标准下的可靠评分，以及 6 步法的实操路径：拆维度→建黄金集→人评+机评→校准→扩集→Bad Case 反推，重点掌握“校准比扩集重要十倍”的工程原则。

引言

阿里云教你评对话——用 AI 评 AI，抓 Bad Case。

腾讯 TPerf 教你评工具——扣分制 + 0 容忍，守住客观底线。

这一节是最难的一类：**内容生成型 Agent**。

AIGC 配图 Agent 的任务是：给一篇文章自动生成配图。

评估标准？“图好不好看”。

“好不好看”是什么意思？10 个人有 11 个答案。这不是代码规则能判断的，这也不像对话质量那样有相对客观的“功能正确性”做锚点——这是纯主观判断。

那到底怎么评？

AIGC 配图团队给了一个答案：**不是找到绝对标准，而是建立可靠的相对评分体系——核心是校准，不是数量。**

💡 内容生成型 Agent 的核心挑战

三类 Agent 的评估本质差异：

类型	标准	核心工具	最大挑战
对话型	半主观（功能+体验）	LLM 裁判	没有唯一正确答案
工具型	客观（步骤+结果）	代码规则	序列比对与容忍度
内容生成型	主观（审美+一致性）	LLM 裁判 + 人工校准	评分本身不可信

注意最后一行：**评分本身不可信**。

对话型评分不可信时，用多 LLM 对抗来降低偏差。但内容生成型的问题更深——什么是“好的配图”，不同领域（科技文章 vs 情感文章）的标准完全不同，同一张图对不同受众的评分可能相差 2 分（满分 5 分）。

这意味着评分器在未经校准的情况下，根本不能用来做决策。

校准，是内容生成型 Agent 评估的核心工作。

六步法：从 0 到 1 搭起整套评估体系

Step 1: 拆维度（避免”好不好看”这种空泛标准）

直接让 LLM 评”这张图好不好”，结果是不可重复的——不同 prompt、不同时间跑出来的分数差异很大，没有参考价值。

正确做法：把”好不好看”拆成可独立评判的维度。

AIGC 配图团队拆出了四个维度：

维度 1：主题相关性	— 图片内容与文章主题是否匹配？
维度 2：风格一致性	— 图片风格与文章调性是否一致？（严肃报道 vs 轻松博客）
维度 3：视觉质量	— 图片构图、清晰度、色彩是否合理？
维度 4：安全合规	— 是否包含敏感内容？（商业场景的刚需）

每个维度独立打分（1-5 分），而不是一个整体分。

为什么要拆？ 因为 LLM 面对”好不好看”这种整体问题时，会被强势维度（比如视觉质量很高）拖带，忽视弱势维度（比如主题相关性低）。独立评分强制 LLM 对每个维度专注思考。

Step 2: 建黄金集（10 个高频场景就能启动）

黄金集（Golden Set）是用来校准评分器的基准数据——每条 case 的”正确分数”经过人工仔细确认，后续所有评分器都对照它校准。

不要等准备好了再开始——10 个高频场景就能启动。

AIGC 配图的黄金集选取原则： 1. 覆盖主要内容类型（科技/情感/资讯/教程） 2. 包含正例（明显好的配图）和反例（明显差的配图） 3. 每个 case 只需要：输入（文章摘要）+ 期望风格 + 配图

黄金集不需要标准答案（生成型任务没有唯一标准答案），需要的是：**当你看到这张图时，你的直觉判断是什么，以及这个判断的理由是什么。**

这个”理由”就是后续校准 LLM 裁判的材料。

Step 3: 人评 + 机评（选择打分体系）

黄金集建好后，同时运行人工评分和 LLM 评分，建立初始对照。

打分体系选择（回顾 Ch9）： – **GSB（Good/Same/Bad）**：适合横向比较（这次生成的图比上次好吗？） – **MOS（整体均分）**：适合跨时间的趋势追踪（这周的配图整体质量上升了吗？） – **Likert（逐维度打分）**：适合精细诊断（到底哪个维度拖了后腿？）

AIGC 配图团队主要用 Likert，因为他们需要精确知道是哪个维度的问题，才能定向优化生成 Prompt。

评分 Prompt 的强制约束：

你是一位专业的图片评审编辑。请按以下步骤评估这张配图：

1. 主题相关性（1-5 分）：[独立评估 + 理由 + 引用文章中的具体内容]
2. 风格一致性（1-5 分）：[独立评估 + 理由 + 引用图片中的具体视觉元素]
3. 视觉质量（1-5 分）：[独立评估 + 理由]
4. 安全合规（通过/不通过）：[是否包含敏感内容]

输出格式（JSON，不允许其他格式）：

```
{
  "relevance": {"score": int, "reason": str, "evidence": str},
  "style": {"score": int, "reason": str, "evidence": str},
  "visual": {"score": int, "reason": str, "evidence": str},
  "safety": {"pass": bool, "issue": str | null}
}
```

“evidence”字段是强制的——强迫 LLM 引用具体内容作为打分依据，而不是空泛结论。这是提高评分可信度最有效的单一手段。

Step 4: 校准 AI 评分（校准比扩集重要十倍）

把 LLM 评分结果和人工标注对比，计算三个对齐指标：

指标 1：各维度分差

人评风格一致性均分：3.8，LLM 评：4.6 → 分差 0.8

判断标准：分差 > 1.0 = 对齐不足，需要调整

指标 2：排序倒置率

人工认为 A > B，LLM 认为 B > A 的比例

判断标准：倒置率 > 20% = 对齐不足

指标 3：极端 case 一致率

人评最高分（5）的 case，LLM 也评高分（4-5）的比例

判断标准：一致率 < 80% = 对齐不足

AIGC 配图团队的实际校准过程：

第一轮：LLM 的风格一致性维度分差 = 0.8（可接受范围），但排序倒置率 = 35%（不对齐）。

问题定位：LLM 在科技类文章和情感类文章上用了同一个“一致性”标准——科技文章配一张有数据图表的图，LLM 打高分；情感文章配同一张图，LLM 也打高分。但人工评分认为情感文章配数据图表是不合适的。

调整：在 Rubric 中增加“先判断文章类型，再用对应类型的一致性标准评估”。

第二轮：排序倒置率从 35% 降至 12%，对齐达标。

这个过程：从发现问题到校准完成，花了 2 天。但它让后续所有 LLM 评分的可信度提升了一倍。

为什么”校准比扩集重要十倍”：

没有校准的评分器，覆盖 500 条样本也是无效的——你收集的是噪声，不是信号。

校准好的评分器，即使只有 10 条黄金样本，每次打出来的分数都是可信的判断依据。

不要在不可信的评分器上扩规模。

Step 5: 扩集到生产规模

校准完成后，开始扩大评测规模：

黄金集（10 条）→ 用于校准
生产集（100-500 条）→ 按用户 query 频率构建，日常评估用

生产集的人工介入比例：

- 5% 样本人工抽检（校准维护）
- 95% 全部 LLM 自动打分

按 query 频率构建的意思是：哪类配图请求最多，生产集中就要有更多这类样本。

如果 80% 的请求是科技文章配图，生产集中科技文章类 case 也应该占多数——这样生产集的分数才能真实反映整体服务质量，而不是被低频场景的高分拉高平均值。

Step 6: Bad Case 反推 + 动态进化

生产集跑起来后，持续做 Bad Case 反推（回顾 Ch26）。

AIGC 配图团队的一个具体例子：

现象：科技类文章中，“概念解释类”文章（比如“什么是量子计算”）的配图主题相关性评分长期偏低。

归因：不是生成的图本身差——是生成 prompt 里缺少对”抽象概念的隐喻化视觉表达”的指导。Agent 生成的 prompt 倾向于字面匹配（“量子”→用量子计算机图片），而不是概念隐喻（“量子叠加态”→用薛定谔猫或多重可能性的图片）。

优化：在文章类型识别逻辑里，增加”概念解释类”的识别，以及对应的隐喻化 prompt 生成策略。

效果：概念类文章的主题相关性评分从平均 2.9 提升到 4.1（满分 5）。

动态进化机制：

黄金集

→ 每月更新一次（纳入新发现的典型 case）

生产集

→ 新模型上线后，回跑全部历史 case（而不是只跑新 case）

评分器

→ 随 Rubric 调整，对历史 Trace 重新评分（参考 Ch28 的重新评分模式）

终极目标

→ 系统能自动识别"什么类型的文章+什么类型的配图=系统性低分"，触发自动优化

三类 Agent 评估横向对比

三章结束了，回顾一下三类 Agent 的评估核心差异：

维度	对话型（阿里云）	工具型（腾讯 TPerf）	内容生成型（AIGC 配图）
标准类型	半主观	客观	纯主观
核心评分器	LLM 裁判	代码规则 + LCS	LLM 裁判（重度校准）
稳定性策略	多 LLM 对抗	0 容忍	校准驱动
最大投入	场景映射 + 上下文工程	步骤序列定义	校准（持续维护）
核心洞察	评估-诊断-优化三段闭环	扣分制暴露薄弱点	校准比扩集重要十倍

本节核心观点

内容生成型 Agent 评估的核心挑战：评分本身不可信——校准是前提，不是优化。

六步法：拆维度→建黄金集（10 条启动）→人评+机评→校准→扩集→Bad Case 反推

三个对齐指标（判断 LLM 评分器是否可信）： 1. 各维度分差 < 1.0 2. 排序倒置率 $< 20\%$ 3. 极端 case 一致率 $> 80\%$

动态进化机制：黄金集每月更新 + 新模型回跑历史 + 评分规则独立迭代

核心原则：校准比扩集重要十倍。10 条校准好的黄金样本，比 500 条未校准的生产样本更有价值。

实操

第一步：参照六步法，为你自己的 Agent 写一份”从 0 到 1 搭建评测系统”的执行计划。每一步列出：– 具体产出（例：Step 1 = 4 个独立可评分的维度定义文档）– 完成时间节点 – 判断该步完成的标准

第二步：针对 Step 4（校准），立刻可以做的最小动作是：– 用你 Ch8 设计的评分 Rubric，对同一批样本（Ch6 的 3 条手工标注）跑一次 LLM 评分 – 计算分差：LLM 的平均分和你的人工标注均分差了多少？– 如果分差 > 1.0 ，找出最大的分歧点，调整 Rubric

第三步：你的评分 Prompt 里有 evidence 字段吗？如果没有，立刻加上——这是提高评分可信度最简单的单一改进。

下节预告

三类 Agent，三种评估体系，三套核心方法。

三章案例讲完了，整门课也到了终点。

最后一章：从你第 1 章的盲测出发，到现在你拥有的完整评估能力——梳理这 30 章的核心转变，给自己一个”毕业”清单，然后走出去，用证据替代运气。

课程总结——从靠感觉到靠证据

学习目标：完成从“靠感觉”到“靠证据”的完整认知升级——回顾 30 章的核心转变链条，整理你的毕业清单，确认你带走了什么、接下来第一步做什么。

引言

30 章（含 Ch0 全景图），~165 分钟。

如果只记住一句话：**Agent 开发的质量不取决于你写了多聪明的 Prompt，而取决于你有多了解它做对了什么、做错了什么。**

回到 Ch1 那个“同一个问题跑 3 次”的你。

那时候的结论是：感觉还行，偶尔会犯一些错误，应该问题不大。

这个结论有什么问题？

- “感觉”不能被复现
- “偶尔”不知道是 5% 还是 40%
- “应该没问题”没有任何证据支撑

你没法用这个结论做决定——上不上线？优化哪里？这版比上版好还是坏？都答不上来。

这就是感觉驱动的 Agent 开发的根本困境：你不知道它有多好，所以你不知道什么时候够好了，也不知道哪里还不够好。

评估不是负担——它是你唯一能信任的镜子。

八步转变链条

这 30 章，走过了一条完整的转变链：

感觉驱动 (Ch1: 同一个问题跑 3 次, “感觉还行”) ↓ **认知驱动** (Ch2-5: 知道 Agent 有哪些失败模式, 知道评估不只是对错) ↓ **方法驱动** (Ch6-11: 手工评估→代码评分器→LLM 裁判→GSB/MOS/Likert→流水线→校准) ↓ **策略驱动** (Ch12-17: 四类 Agent 分类、E2E vs 分层、线下 vs 线上、能力 vs 回归、打分 vs 对比、Swiss Cheese + 分级准出) ↓ **工程驱动** (Ch18-22: 测试集工程、用例设计、基线管理、Trace 分析、Anthropic 8 步路线图) ↓ **进阶驱动** (Ch23-26: 上下文工程、深度思考、pass@k/pass^k、Bad Case 反推) ↓ **案例驱动** (Ch27-29: 阿里云对话型、腾讯工具型、AIGC 内容生成型——三类真实落地) ↓ **证据驱动** (你现在所在的位置)

每一步不是推翻上一步, 而是在上一步的基础上加了一个维度。

✓ 毕业清单

你的 Agent 评估系统达到什么程度, 才算”毕业”?

基础层 (必须): - [] 有 10+ 条经人工确认的 Ground Truth 用例 - [] 有至少 2 条自动运行的代码规则 (格式/边界检查) - [] 有 1 个校准过的 LLM 裁判 Rubric (Spearman 相关性 > 0.7) - [] 有 Trace 记录 (每次运行可追溯) - [] 区分了能力评估和回归评估阶段

工程层 (应该): - [] 三层流水线可自动运行 (代码→LLM→人工仲裁触发条件) - [] 评估环境有隔离 (每次 Trial 从干净状态启动) - [] Bad Case 有分类管理 (按失败模式, 不是按单个 case) - [] 有明确的准出标准 (L1/L2/L3 对应不同上线条件)

进阶层 (有更好): - [] pass@k 和 pass^k 都在监控 - [] 线上失败案例自动回流到线下测试集 - [] 月度帕累托分析 (Top 失败模式是否在变化, 参考 Ch26 Bad Case 反推方法) - [] 校准周期建立 (多久重新校准一次 LLM 裁判)

🧰 你带走了什么

一套工具箱:

- 任务定义: Task 三要素 (输入、工具、成功标准)
- 评分器三层: 代码评分器 / LLM 裁判 / 人工标注
- 分值体系: GSB (对比) / MOS (绝对) / Likert (多维度)

- 统计工具：pass@k（峰值能力） / pass^k（工程稳定性）
- 防御体系：Swiss Cheese 多层防御
- 准出机制：L1/L2/L3 分级，不同层级不同标准
- 反推方法：Bad Case 三步法（归类→帕累托→优化）
- 路线图：Anthropic 8 步，今天就能开始

一个贯穿案例：

客服 Agent 物流查询——从 Ch1 的第一次盲测，到 Ch26 的 Bad Case 反推，这个 Agent 被拆解、被测量、被优化了 30 次。你对这个 Agent 比对你的工程直觉更了解。

三个参照系：

- 阿里云：评估-诊断-优化闭环，对话型 Agent 的完整运营体系
- 腾讯 TPerf：扣分制 + 0 容忍，工具型 Agent 的严苛标准
- AIGC 配图：6 步法 + 校准驱动，内容生成型 Agent 的对齐方法

四个关键转变

转变一：从“跑通一次”到“统计证据”

跑通一次不是证明，是采样。同一个任务跑 10 次，计算 pass@k 和 pass^k，才是对 Agent 能力的有效陈述。非确定性系统需要统计性结论，不是点估计。

转变二：从“感觉变好了”到“可对比的分数”

“感觉这版比上版好”没有信息量。“这版的 GSB 胜率是 73%，上版是 61%，提升显著”才有信息量。可对比的分数让你知道：好在哪里、好多少、值不值得上线。

转变三：从“评估是验收”到“评估是迭代引擎”

把评估放在最后做，是在等待失败。评估放在开发循环里，是在用数据驱动每一次迭代。从“写 Prompt → 看效果 → 凭感觉改”升级为“写 Prompt → 跑测评 → 读数据 → 有针对性地改”。

转变四：从“修单个 Bad Case”到“反推系统性问题”

一个一个修 Bad Case，修到第 5 个会发现前 3 个的修复互相冲突。把 50 个 case 摊在桌上，找失败模式，找 Top 20% 的系统性问题，修一个消灭 20 个——这是质量工程的正确姿势。

⚠ 永远记住这一条

永远不要依赖单一评估方法。

代码评分器有盲区（抓不到语义），LLM 裁判有偏差（近期偏见/评委效应），人工评估有疲劳（标注一致性随时间漂移）。

三层结合：代码评分器处理客观逻辑，LLM 裁判处理语义质量，人工抽检校准对齐——每一层补充另一层的盲区。没有完美的评估方法，只有互补的评估体系。

🎯 课程核心观点总结

认知层：

Agent 评估的本质，是把”感觉这版还行”变成”这版在 87 个测试任务上的通过率是 91.3%，比上版提升 8.2%，主要进步来自意图识别类任务，在多轮上下文场景仍有待提升”。

方法层：

没有标准答案时用 GSB 对比；有参考解时用 MOS/Likert 打分；有基准路径时用 LCS 比对；都有时叠加三层。方法选择跟着场景走，不跟着习惯走。

工程层：

测试集是资产，要版本控制。评分器是工具，要持续校准。Trace 是证据，要认真读。Bad Case 是信号，要系统反推。评估套件是基础设施，不是一次性项目。

哲学层：

评估不是给 Agent 打分，是在和 Agent 对话——它告诉你它做错了什么，你告诉它应该怎么做。这个对话永远不会结束，但每一轮都让 Agent 更可靠一点。

📖 课程知识图谱



贯穿始终的一个案例：**客服 Agent**，一个案例跑完 30 章。

贯穿始终的一句话：“你改的不是代码，你改的是概率分布。”

📌 下周计划模板

拿出纸笔，花 2 分钟填完这三栏：

我的 Agent 类型	最薄弱的环节	下周具体行动
(对话型/工具型/内容生成型/混合)	(基础层/工程层/进阶层，参考毕业清单)	(一个具体的、可完成的任务)

今天下班前的三件事

第一件：用毕业清单自测——各层缺了哪些项？找出最紧急的缺口。

第二件：选定下周要填补的最高优先级缺口，写下具体的行动方案。

第三件：将评估纳入日历——设定每月固定的 Bad Case 回顾和 Trace 抽检时间。评估是持续工程，不是一次性验收。

感谢你完成了这 30 章。

从同一个问题跑 3 次开始，到现在——你已经不是那个只能靠感觉的人了。评估不是给 Agent 打分，是在和 Agent 对话。这个对话永远不会结束，但每一轮都让 Agent 更可靠一点。